BRIEFING DECK · COMPUTE-PAUSE VERIFICATION

Research directions for verifiable AI slowdown

A self-contained technical map of the main public papers and posts on verifying (and preferably throttling) frontier AI training under low trust. For each piece: who wrote it, what problem it addresses, how mature it is, and how it relates to the rest of the stack. Diagrams first; public links in each section.

2026-08-04 · updated 2026-11-12 (§0c complete crypto foundations for §§6–8, §10) · standalone briefing · sources linked inline

ORIENTATION

0. Why this exists, what "Claim A" means, and the stack

0.0 Motivation — why anyone is building this

Anthropic (2026), *When AI builds itself*. The Institute argued the world should have the *option* of a coordinated slowdown or temporary pause on frontier development — but only if parties can **verify** that others actually stopped, so bad actors cannot use a collective pause as cover to race ahead. Conditional pause language is the point: verification systems are a prerequisite, not a side quest. (anthropic.com/institute/recursive-self-improvement)

AI 2027, Appendix S (scenario research). If the US and China tried an AI arms-control style deal, how would cheating be caught? The appendix lists four imperfect families: (1) intelligence collection, (2) crude compute shutdowns with inspectors, (3) **hardware-enabled mechanisms (HEM)** — tamper-resistant reporting of what accelerators run, analogous to seals/attestations in other verification regimes, and (4) politically extreme "AI lie detection" of officials. This deck is mostly fleshing out family (3) and the monitoring stack that sits under it. (ai-2027.com · research notes on compute / verification)

Same problem, different genres: a lab institute essay (Anthropic) and a public scenario appendix (AI 2027) both treat **credible verification** as the binding constraint on any pause story.

0.1 The policy problem in one paragraph

Labs and governments sometimes talk about a **coordinated slowdown or pause** on frontier model development. That option is worthless if parties cannot tell whether others actually stopped. Self-reports are not enough under low trust (rival states, or competing labs). The technical agenda is: build monitoring / attestation / enforcement so large unauthorized *training* is discoverable or physically hard — without requiring mutual trust in each other's word.

Training vs inference is the load-bearing distinction. **Training** updates weights (often with heavy multi-GPU communication). **Inference** serves a fixed model. Datacenters can look "busy" either way; a sophisticated actor can try to disguise training (especially post-deployment RL) as serving. So "the GPUs are warm" is not a proof of compliance.

0.2 Four claims people mix up

People say "AI verification" for several *different sentences*. This deck is almost entirely about **Claim A**.

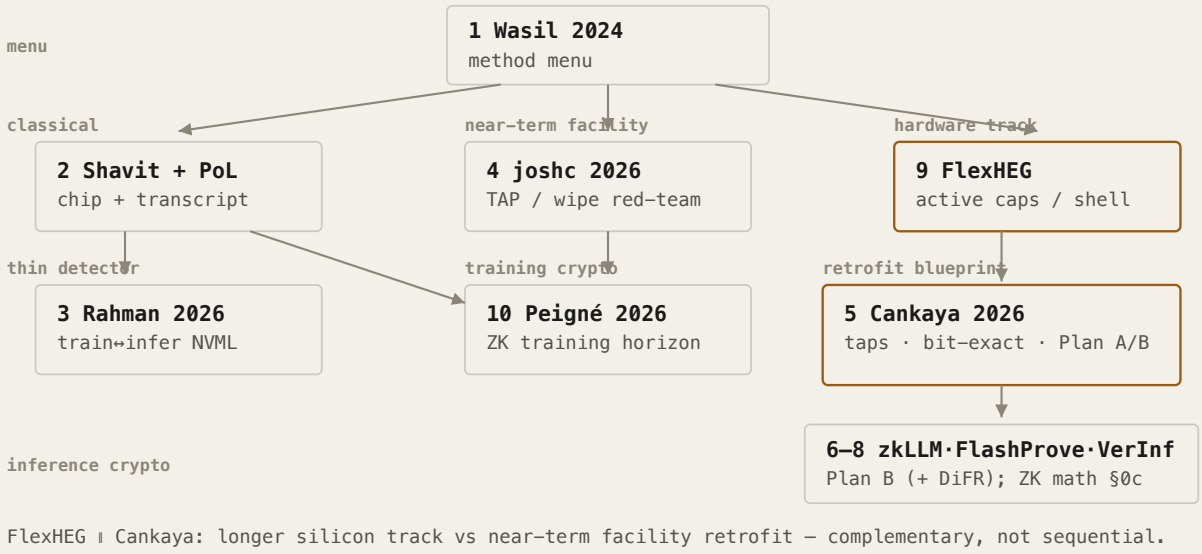


Mixing claims is the most common briefing failure. FlexHEG / Cankaya / joshc / Rahman are mostly Claim A. Commercial TEE "AI Passport" products are mostly B/C (sometimes D) — useful, but they do not by themselves prove "no covert training."

CLAIM	PLAIN ENGLISH	PRIMARY AUDIENCE	EXAMPLE TECH
A	No covert frontier training; only allowed / declared workloads on monitored compute	Treaties, pause/slowdown, export-control regimes	TAPs, recompute, train↔infer detectors, FlexHEG, PoL-style transcripts
B	The model that answered you is the one you bought / audited	Enterprise buyers, regulated sectors	Model identity binding, inference receipts, DRM-adjacent ZK
C	This run happened in a stated place / TEE; the host can't read your data	Sovereignty, GDPR, confidential cloud	Confidential computing, remote attestation, location proofs
D	This agent only performed approved tool calls	Agent products / enterprise copilots	Audit logs, policy engines, "passport" receipts

> SCOPE OF THIS DECK
>
> **In:** Claim A mechanisms and the research lineage around them. **Out (except as contrast):** commercial TEE inference products aimed at B/C/D. Those can share primitives (attestation, recompute) but answering B/C does not answer "did you secretly train a bigger model?"

0.3 Map of works covered



Wasil orients. Shavit designs classical monitoring; Rahman / Peigné harden thin slices of it (telemetry vs ZK transcripts). joshc stress-tests cheap facility controls → Cankaya assembles the retrofit; zkLLM / FlashProve / VerInf / DiFR sit under Plan B evaluate (§0c = ZK math foundations). FlexHEG is the parallel hardware-enforcement track.

loose	**NTM / inspections** — satellites, energy, site visits (Wasil’s access-light methods)
medium	**Network / facility controls** — TAPs, bandwidth caps, memory wipes, output recompute (joshc; Cankaya taps)
tight	**Workload monitoring** — weight snapshots + transcripts (Shavit); thin train↔infer telemetry (Rahman)
hard	**Hardware-enabled guarantees** — FlexHEG FLOP caps / licenses / interlocks
crypto	**ZK training proofs** — Peigné et al. (§10 research horizon)
infer	**Near-term inference retrofit** — Cankaya Plan A (§5 taps + bit-exact) then Plan B (§6 zkLLM · §7 FlashProve · §8 VerInf; algorithms in §0c)

Maturity legend used below: *Survey* (conceptual menu) · *Blueprint* (architecture, no full deploy) · *Empirical paper* (measured results) · *Prototype / demo* · *Horizon* (no working end-to-end system yet) · *Product* (mass deployment — almost nothing here qualifies).

PRIMER

0b. Hardware · ML systems · crypto vocabulary

Zoom from hall → interconnect layers → rack → node → board → SM, then why traffic looks different for train vs infer, then a short ZK vocab teaser. Full ZK math is §0c. Skim once; jump back when a later section feels opaque.

1 · Where this lives — a real hall



Real AI cluster hall (Meta AI Research SuperCluster press photo): rack rows, cold-aisle containment, overhead cable trays carrying the scale-out fabric. Source: Meta AI — Introducing the AI Research SuperCluster.

2 · Organizing abstraction — interconnect ladder

Everything below is a zoom into one of these layers. Tap feasibility is uneven because only some layers ever become light.



IB/RoCE (layer 3) carry the cross-node all-reduces Claim A people want to meter. Layer-2 copper spines move TB/s without becoming light — hard tap case for Cankaya/joshc.

LINK	MEDIUM	JOB	TAP NOTE
NVLink / NVSwitch	Copper (board / cable cartridge / rack spine)	Scale-up: activations, TP shards, in-rack collectives	Hard — never becomes light
InfiniBand (Quantum…)	DAC copper short; fiber + QSFP/OSFP longer	Scale-out GPU↔GPU (NCCL over RDMA)	Fiber segments = tap candidates
RoCE / Spectrum-X	Same physical story as IB, Ethernet framing	Same role as IB in many clouds	Same EO story
PCIe	Board traces	GPU↔CPU / GPU↔NIC	Inside box
AOC / transceiver	Electrical→laser→fiber→photodiode→electrical	Longer runs, leaf–spine, NS exit	Conversion lives in the pluggable module

Electro–optical conversion in one sentence: the NIC SerDes is electrical; for distance you plug in a module that drives a laser into fiber and recovers bits at the far end. Passive optical taps copy photons on the fiber; they do nothing for copper NVLink spines inside the rack.

3 · Ladder mapped onto a facility

Datacenter hierarchy — IB/RoCE ≠ "only cross-pod"

GPU + HBM
die · SMs
ladder layer 1

Node (e.g. 8-GPU)
NVLink inside = layer 2
NIC = layer-3 edge

Rack
many nodes, or
1× NVL72 domain

Pod / cluster
scale-out fabric
IB/RoCE leaf-spine

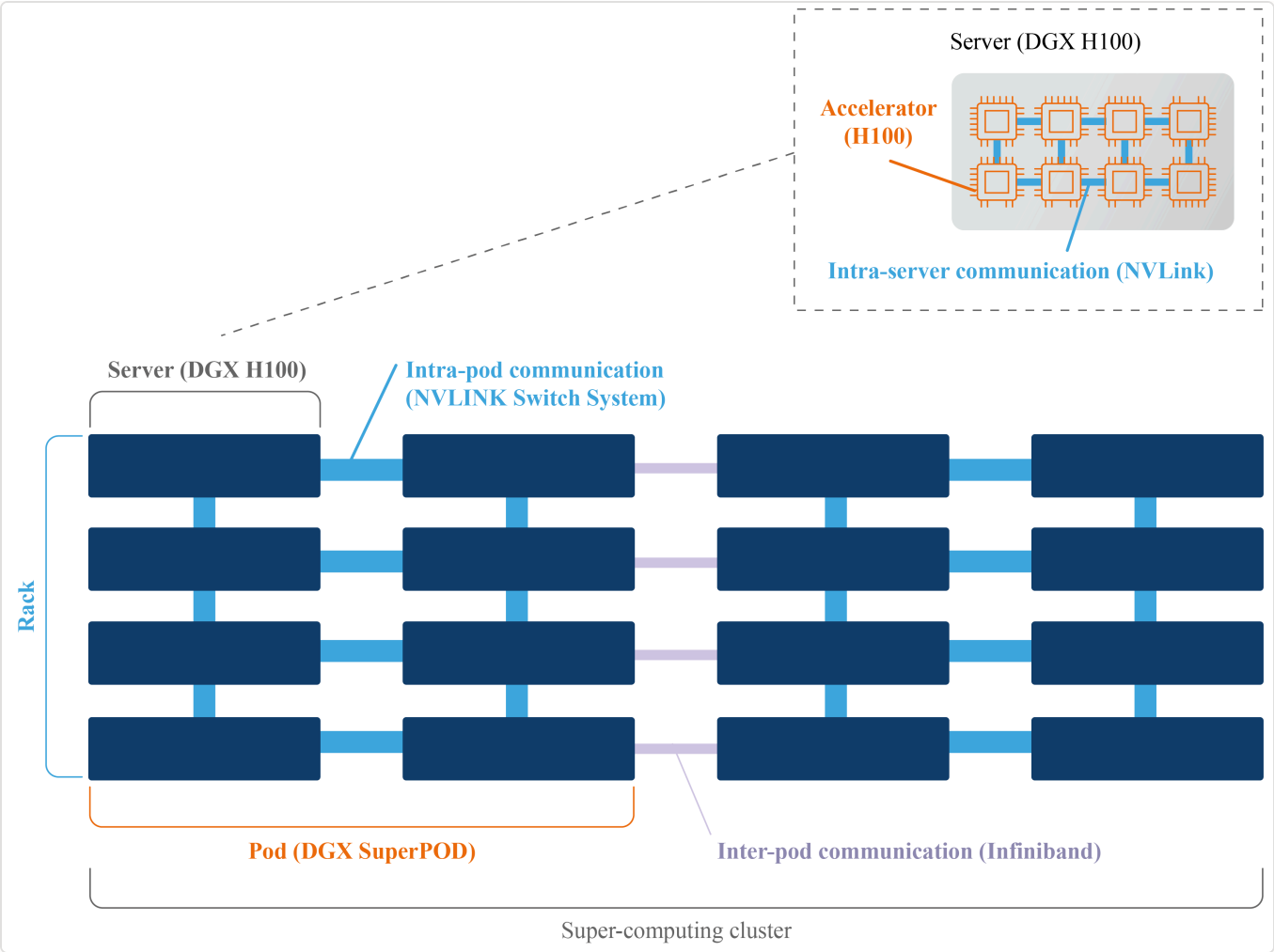
Where IB/RoCE actually starts = leaving the NVLink domain

Classic DGX: NVLink = one 8-GPU node → even same-rack node↔node is already IB/RoCE.
NVL72: NVLink = whole rack (72 GPUs) → IB/RoCE starts rack↔rack (in-pod or cross-pod).

East–West (inside the fence)
GPU↔GPU grads / acts / KV
= NVLink (in-domain) + IB/RoCE (out)
Fat · hard to tap fully

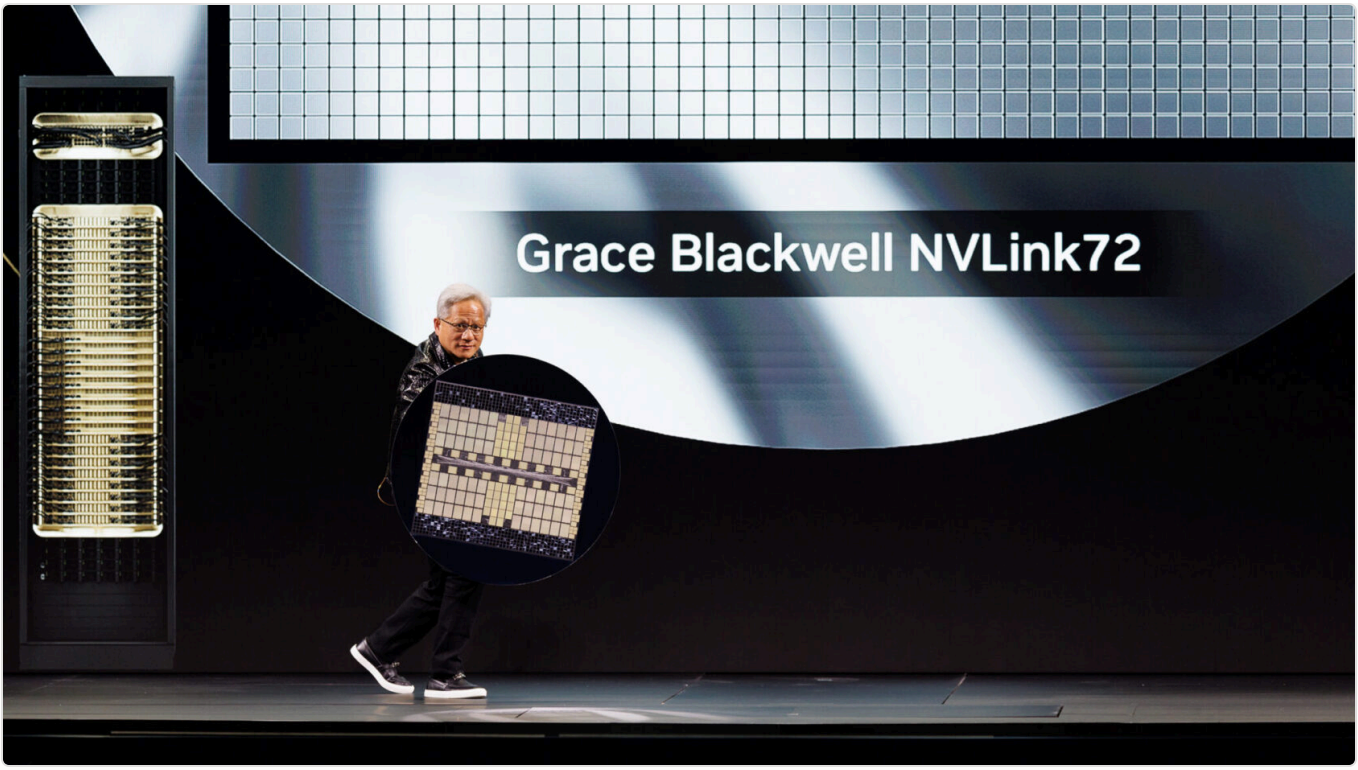
North–South (to the outside)
API tokens, SSH, admin, uploads
Skinny · natural chokepoint
Best first place for taps

NIC sits on the node because that is the scale-out edge for classic topologies — not because IB is "a pod thing." Cross-pod is just one hop on the same scale-out fabric.



Same hierarchy, drawing form: servers → racks → pods → site. East–West ≈ ladder layers 2–3 inside the fence; North–South exits the site (best first tap chokepoint).

4 · Zoom: one scale-up domain (rack)

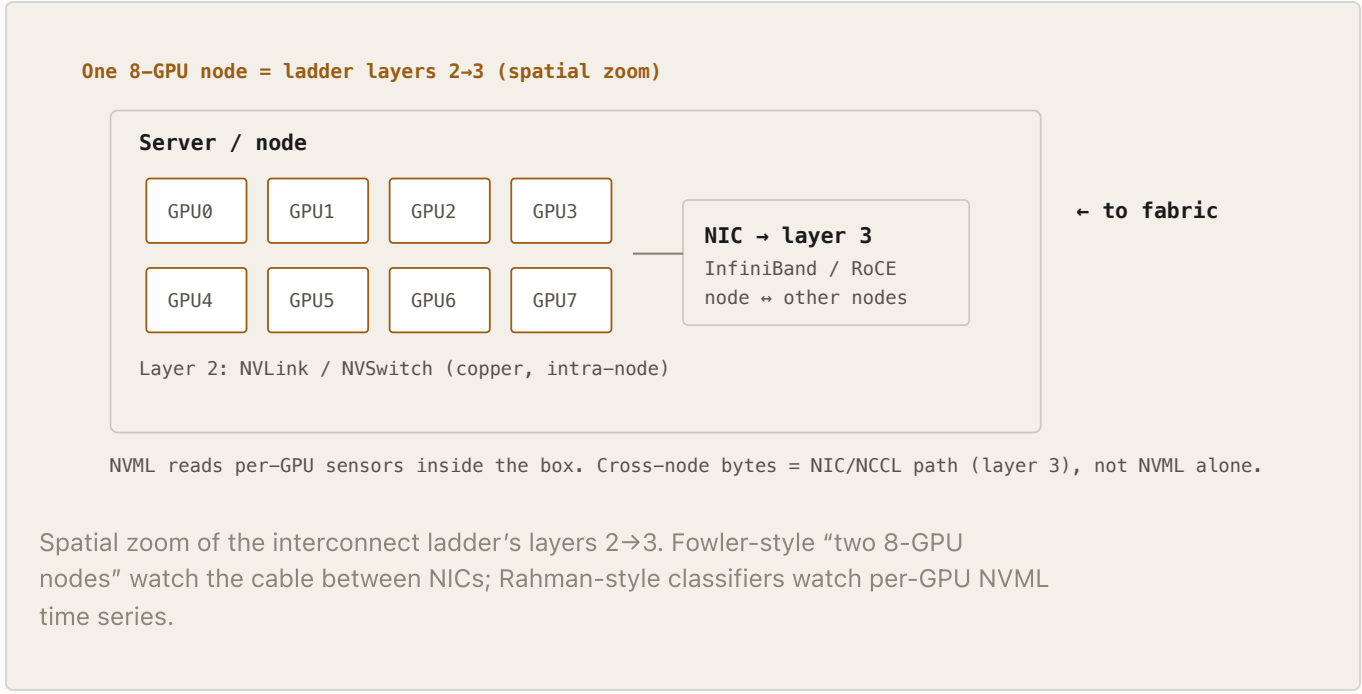


Ladder layer 2 at modern density: GB200 NVL72 = 72 Blackwell GPUs + Grace CPUs in one liquid-cooled copper NVLink domain (~130 TB/s). Source: NVIDIA Blog.



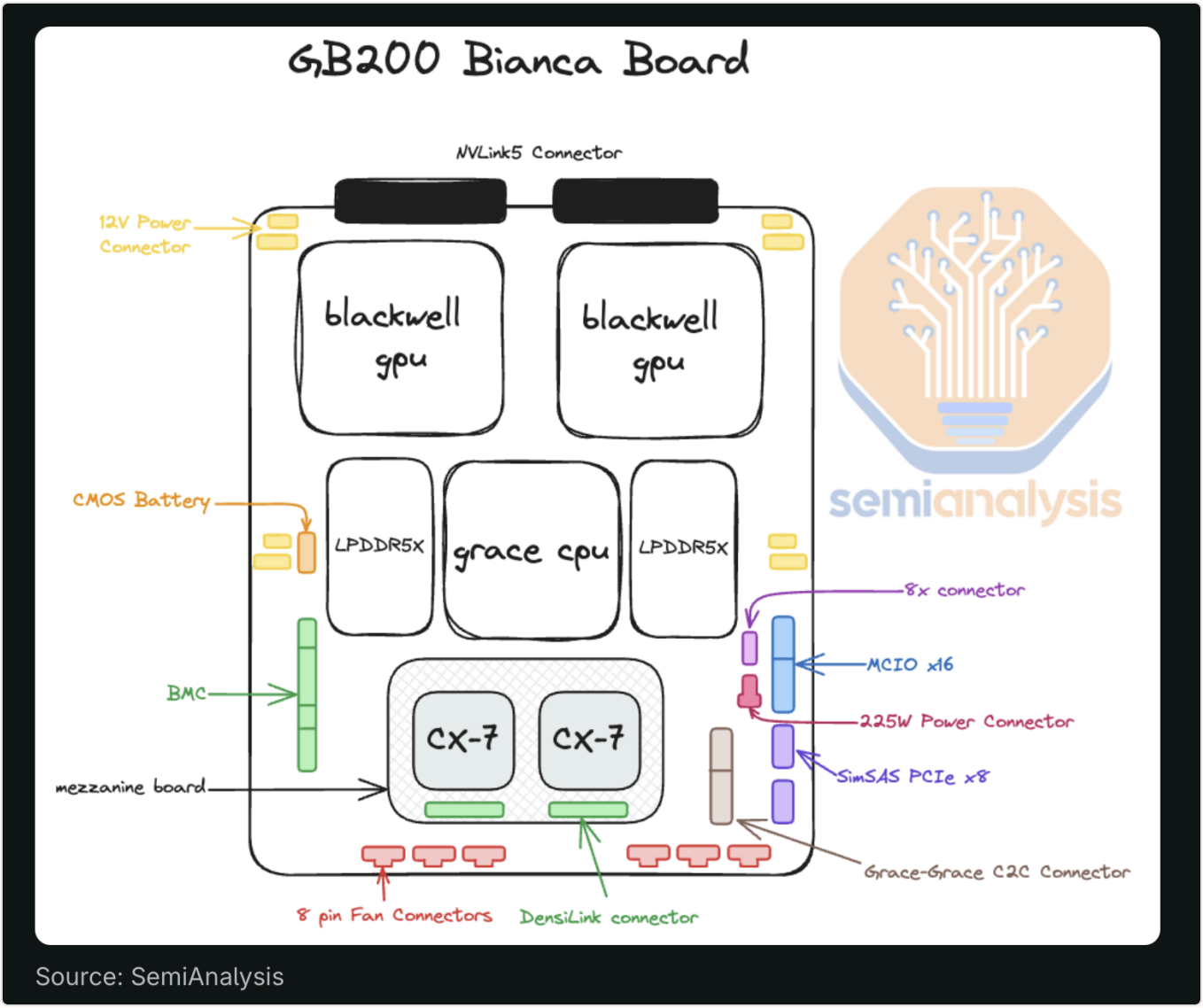
What layer-2 cabling looks like up close: dense copper (not fiber). Optical conversion mostly starts at the scale-out NIC edge (layer 3). Source: NVIDIA Blog.

5 · Zoom: one node (ladder layers 2→3)



Spatial zoom of the interconnect ladder's layers 2→3. Fowler-style "two 8-GPU nodes" watch the cable between NICs; Rahman-style classifiers watch per-GPU NVML time series.

6 · Zoom: one board (ladder layers 1→2)



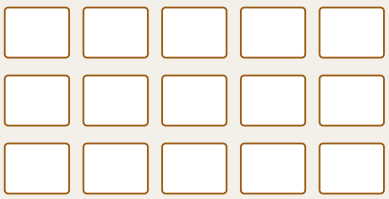
Source: SemiAnalysis

GB200 “Bianca” board (SemiAnalysis). Layer-1/2 callouts: **Blackwell ×2** — AI compute dies (SMs + HBM on-package); **Grace CPU** — Arm host / orchestrator; **LPDDR5X** — Grace system DRAM (not GPU HBM); **NVLink-C2C** — Grace↔GPU coherent link (still layer 1); **NVLink5 connectors** — out to rack scale-up domain (layer 2); **CX-7** — NIC mezzanine → IB/Ethernet scale-out (layer 3; EO starts here on long runs); **Grace–Grace C2C** — board-to-board CPU link; **BMC** — out-of-band management; **MCIO / SlimSAS** — PCIe storage/expansion. FlexHEG near-term retrofits often target the *NIC / enclosure*, not a GPU-die respin.

7 · Zoom: inside the GPU die — SM

SM = Streaming Multiprocessor (GPU's compute tile)

One GPU die (cartoon)



H100 ≈ 132 SMs · each SM = many CUDA cores + Tensor Cores + schedulers

What "SM util" actually measures

NVML "SM utilization" ≈ fraction of SMs that had ≥1 warp active over the sample window.

High SM util ⇒ compute units busy.
Low SM util + high HBM BW ⇒ memory-bound (classic decode / bandwidth-limited kernels).

Does **not** say train vs infer, legal vs covert, or which model. Just "are SMs awake?"

Think of SMs as many small CPUs glued to shared HBM. Prefill matmuls wake lots of SMs; token-by-token decode often waits on memory → SM util can look "idle" while the GPU is still serving.

8 · Why so many GPUs, and why traffic differs

Frontier training is big because the *model state does not fit on one GPU*, so you must shard and communicate constantly — mostly on ladder layers 2–3.

SCALE INTUITION (PUBLIC ANCHORS, ~2024–26)	NUMBER
Llama 3.1 405B dense params	405×10^{9}
BF16 weights alone	405B × 2 B ≈ 810 GB
Naive train state (W + grad + Adam ≈ 16 B/param)	≈ 6.5 TB total
One H100 HBM	80 GB (SXM) / ~94–96 GB (some SKUs)
B200 / Blackwell HBM class	~180–192 GB/GPU (gen-dependent)
One GB200 NVL72 rack	72 GPUs · ~13.4 TB HBM aggregate
Llama 3.1 405B pretrain cluster (Meta report)	up to ~16,384 H100s · $\sim3.8\times10^{25}$ FLOP · ~15.6T tokens
Rough "why many GPUs": fit train state + activations	405B needs ≫1 GPU; production runs use 10^{3}–10^{4}+

Closed frontier labs (OpenAI / Anthropic / Google) don't publish exact GPU counts; treat Llama 405B as the best public lower-bound picture of "what a near-frontier dense run looks like," not the ceiling. MoE frontier runs can advertise huge total params with fewer active params/token — still multi-thousand-GPU jobs.

Collectives that make training "loud" on the network

All-reduce (DDP heartbeat)

Each GPU has local gradient g_i
→ everyone gets mean(g)
Bytes / step ≈ O(model size)

Big, regular, every step

Cross-node → IB/RoCE counters spike

All-gather (FSDP / some TP)

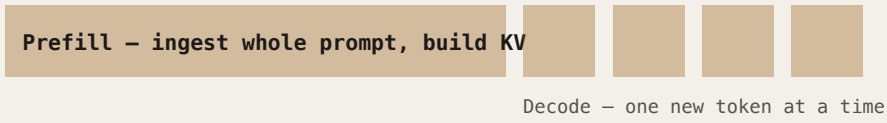
Each GPU holds a shard
→ assemble full tensor locally
Also O(params) class traffic

Inference usually ships activations

or KV — O(tokens×hidden), not O(params)/step

DiLoCo / LocalSGD: train locally for many steps, sync rarely → average cross-node GB/s can look "inference-quiet" while training still happens.

Inference anatomy: prefill vs decode (and why disagg ships KV)



Same box (usual serving)

Prefill+decode share GPUs

Cross-node often quiet

Disaggregated serving

Prefill pool → ship KV → decode pool

Can make NS/EW look "training-loud"

SM util: how busy GPU compute units are. Prefill can pin SMs; decode is often memory-bandwidth bound — so SM util alone is a weak train/infer label.

TERM	MEANING
TP	Tensor parallelism — split a layer's matrices across GPUs; exchange activations each layer
DP / DDP	Data parallel — full model replica per worker; all-reduce gradients
FSDP	Fully sharded data parallel — shard params; all-gather when needed
NCCL	NVIDIA collective communications library implementing all-reduce etc.
NVML / DCGM	Per-GPU sensor APIs (power, SM util, memory…). Not cross-node byte counters
SM util	Fraction of SMs busy in the sample window — compute busyness, not "is this training?"
Warden	Active network scrubber on the monitored path (normalize headers/timing to kill stego) — see §4

TERM	MEANING
DiLoCo	Low-communication training: many local steps, rare outer sync — can evade “fat interconnect” detectors

9 · Zero-knowledge proofs (for §§6–8, §10)

Four properties (use these words precisely in §§6–8)

Completeness
Honest compliant prover almost always passes. False reject → kills the deal. (“If I’m clean, I get accepted.”)

Soundness
Cheater almost never passes. False accept → fake compliance. (“Dirty run can’t forge Accept.”)

Confidentiality
Weights / prompts / activations / customer data stay private from the other party (and often from each other’s silicon). ZK ≈ cryptographic confidentiality of witness.

Assurance
How sure is the auditor that cheating would be caught in the field? Includes soundness + sampling rate + tap coverage + stego slack (joshc ~1%).

Plan A vs Plan B on these axes
Plan A (plaintext AE recompute): can be strong on completeness/soundness/assurance if bit-exact + dense samp
but burns confidentiality (weights + challenged I/O enter the AE). Plan B (ZK): keep soundness/completeness
restore confidentiality of the witness; assurance still needs sampling + good taps — π doesn’t watch untappe

Don’t collapse these. A demo can be complete+sound in crypto and still give low assurance if you sample 0.001% of requests or leave EW copper blind.

ZK vocabulary (enough to read zkLLM / VerInf / Peigné)

Statement

"I ran model W on x and got y" (or a bound)

Public: arch, C=Commit(W)

Witness

Secret intermediates: weights, activations…

Prover holds; verifier doesn't

Constraints

Algebraic equations that every correct step must satisfy (the "circuit")

Properties you want

Completeness: honest prover convinces
Soundness: cheater almost never passes
Zero-knowledge: proof leaks ~nothing extra

In this literature

zkLLM: prove exact integer circuit
VerInf: prove unexplained bits ≤ U
Peigné: ZK for training steps (horizon)

The integer / finite-field constraint (why float GPUs ≠ ZK circuits)

SNARKs do arithmetic in a prime field $\mathbb{F}_p$ (integers mod p). GPU serving does BF16/FP8 floats with rounding. So zkLLM-style systems re-express the forward pass as a quantized / integer circuit, then prove that circuit
What you proved: "integer program with Commit(W) produced y." What you did **not** prove: bit-exact match to arbitrary float serving on H100/B200. Softmax/nondeterminism → lookups + tolerances, not native IEEE floats.
VerInf dodges this by proving an unexplained-info bound instead of exact float equality.

A "constraint" is a math check over $\mathbb{F}_p$ (e.g. $c - a \cdot b = 0$), not a governance policy. The integer gap is why "ZK receipt" and "bit-exact AE recompute" are different Plan B / Plan A stories.

Constraint system intuition: turn "this forward pass was correct" into a huge set of algebraic checks over a *prime field* $\mathbb{F}_p$ (multiplications, lookups, range checks). The prover knows a *witness* (weights, activations) satisfying every constraint; the verifier checks a short proof that such a witness exists for the public statement — without learning the witness.

Integer constraint, concretely: GPUs run BF16/FP8 with rounding; SNARK circuits run integers mod p. zkLLM-style systems therefore prove a *quantized integer circuit* that approximates the model, not "whatever your CUDA float kernel did today." That is why Plan A cares about bit-exact float recompute, and why VerInf replaces exact equality with an unexplained-info bound (Û / U-hat). Failures of soundness = false compliance; failures of ZK / confidentiality = IP/data leakage through the proof; failures of assurance = you never sampled the cheat.

Next: §0c teaches the full ZK toolbox for this deck (shared floor + hash stack + Hyrax/PCS + statement choice + zkVM), each tool tagged with which direction uses it. §§6–8 and §10 apply that toolbox; they are not where the math is introduced.

PRIMER

0c. Zero-knowledge proofs · math foundations

Complete crypto homework for every ZK direction in this deck: §6 zkLLM, §7 FlashProve, §8 VerInf, §10 Peigné. Not a walkthrough of one paper. §0b was hardware; this is the math. Every tool starts with the everyday problem it solves, then says which directions use it.

0 · The story in one picture

USED BY all ZK §§ | §6 zkLLM | §7 FlashProve | §8 VerInf | §10 Peigné

Imagine your friend claims: "I did a huge secret calculation correctly, and the public answer is Y." You are not allowed to see the secret numbers. You also do not want to redo the whole calculation yourself, because it would take forever.

So your friend must somehow **convince** you. That is what a zero-knowledge proof is for. In jargon:

- The friend is the **prover**.
- You are the **verifier**.
- The secret numbers are the **witness** (for AI papers: often model weights and intermediate activations).
- The public claim is the **statement** (for AI papers: often "this committed model, on this prompt, produced these tokens" — or a weaker score; see §17).

You want four things at once:

- **Completeness:** if your friend is honest and the claim is true, you accept.
- **Soundness:** if the claim is false, a cheating friend almost never fools you.
- **Zero knowledge:** after you accept, you still learn almost nothing about the secret numbers beyond the public claim.
- **Succinctness:** checking the proof is much cheaper than redoing the calculation.

This deck's systems do *not* all use the same crypto packaging. They share a floor (fields, random quizzes, equations), then branch into three stacks:

Three crypto stacks this briefing actually uses

A · Hash / IOP

Reed–Solomon + mask
Merkle openings
sumcheck · lookups
Fiat–Shamir

§7 FlashProve

(§8 often same family)

B · Algebraic PCS

Elliptic-curve group
Hyrax / IPA-style
ProveEval at points
+ sumcheck / lookups

§6 zkLLM

Hyrax binds W → C

C · zkVM

Prove a program run
(CPU/GPU step as
bytecode), not a
hand-built ML circuit

§10 Peigné

training horizon

Shared floor

Fields · Schwartz–Zippel · arithmetization · Fiat–Shamir · "what statement?" — then pick stack A, B, or C.

If you only learn stack A, zkLLM's Hyrax and Peigné's zkVM will feel like aliens. Learn the shared floor, then the branch.

Read §§1–3 and §13–14 first (shared floor). Then either the hash branch (§§4–12), the algebraic-PCS branch (§§15–16), the statement-design section (§17), or the zkVM branch (§18), depending on which paper you open next.

1 · Finite fields — "we need clean number rules"

USED BY all ZK §§ | §6 | §7 | §8 | §10

Problem: computers usually do floating-point math (like 0.1 + 0.2), and different GPUs can round differently. Proofs need exact yes/no algebra, not "close enough."

What we do: pick a huge prime number p, and do all addition and multiplication "on a clock with p hours." After you pass p, you wrap around. People write this number system as $\mathbb{F}_p$ and call it a **finite field**.

Why it matters: in a field, every nonzero number has a multiplicative inverse, and polynomials behave nicely. That is why almost every modern SNARK lives in a field, not in IEEE floats.

Catch: if a value grows bigger than about p/2, wrap-around no longer matches ordinary integer arithmetic. So honest systems keep numbers small (range checks / lookups). AI papers often prove an *integer / quantized* model for this reason — or, like VerInf, they change the *statement* so they do not need bit-exact float equality.

Two tiny math facts everything else sits on

1 · Finite field $\mathbb{F}_p$

Pick a big prime p. All + − × happen "mod p" (clock arithmetic).

SNARK math only works here — not in floating-point GPU numbers.

Keep values small so mod p ≈ integers.

2 · Schwartz–Zippel

A nonzero polynomial of degree d is zero at a random point with probability ≤ d / |𝔽| (tiny).

So: check one random point ≈ check the whole polynomial.

Why you care

Sumcheck, batching many claims, "did the prover lie?" — almost all soundness bounds are just Schwartz–Zippel

If you only memorize two formulas from this whole primer: "mod p" and "random point catches a lying polynomial."

2 · Schwartz–Zippel — "one random quiz question catches a fake essay"

USED BY all ZK §§ | sumcheck | batching | Fiat–Shamir quizzes

Problem: checking a huge algebraic claim cell-by-cell would take forever. You need a cheap random test that still catches lies.

What the lemma says, in plain words: suppose someone hands you a nonzero polynomial of degree at most d. If you plug in a random field element, the chance you accidentally hit a root is at most d / p. When p is enormous, that chance is tiny.

What it is used for: almost every "pick a random challenge and check one equation" step in ZK (sumcheck, batching many claims, many Fiat–Shamir challenges, Hyrax evaluation points) is Schwartz–Zippel wearing a trench coat.

Watch this video to better understand: https://www.youtube.com/watch?v=wNghCHYR6Vo

3 · Arithmetization — "turn the computation into equations"

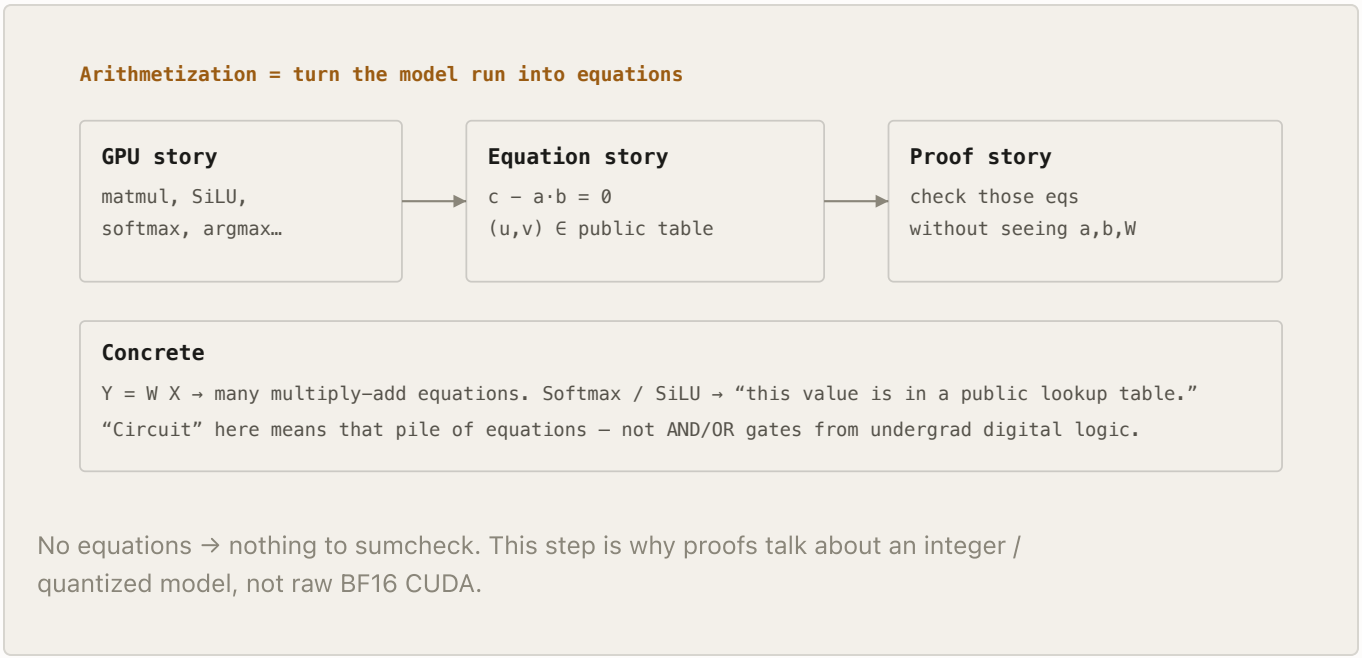
USED BY all ZK §§ | §6 hand-built ML circuit | §7 same | §10 via zkVM trace

Problem: "I ran a neural net" is not a mathematical object you can quiz randomly. You need a pile of exact equations that are true if and only if the run was correct.

What we do: rewrite the computation as many field equations. A multiply becomes "c − a·b = 0." A weird nonlinear function (like SiLU) becomes "the pair (input, output) appears in a

public table." That pile of equations is what people call the **circuit** — not AND/OR gates from a digital-logic class, just equations.

What it is used for: every later tool (sumcheck, lookups, linear claims, Hyrax openings) checks pieces of this equation pile. No arithmetization → nothing to prove. In a zkVM (§18) the "circuit" is the VM's transition rules applied to a long execution trace — same idea, different packaging.



4 · The three-layer sandwich — "math game → hash trick → one file"

USED BY §7 FlashProve §8 VerInf (typical backend family) not Hyrax's commit layer

Problem: you want a short checkable proof, but the honest story of the calculation is enormous. You also cannot sit on a video call tossing random coins forever.

Hash-based systems (stack A in the picture above) usually build the answer in three layers:

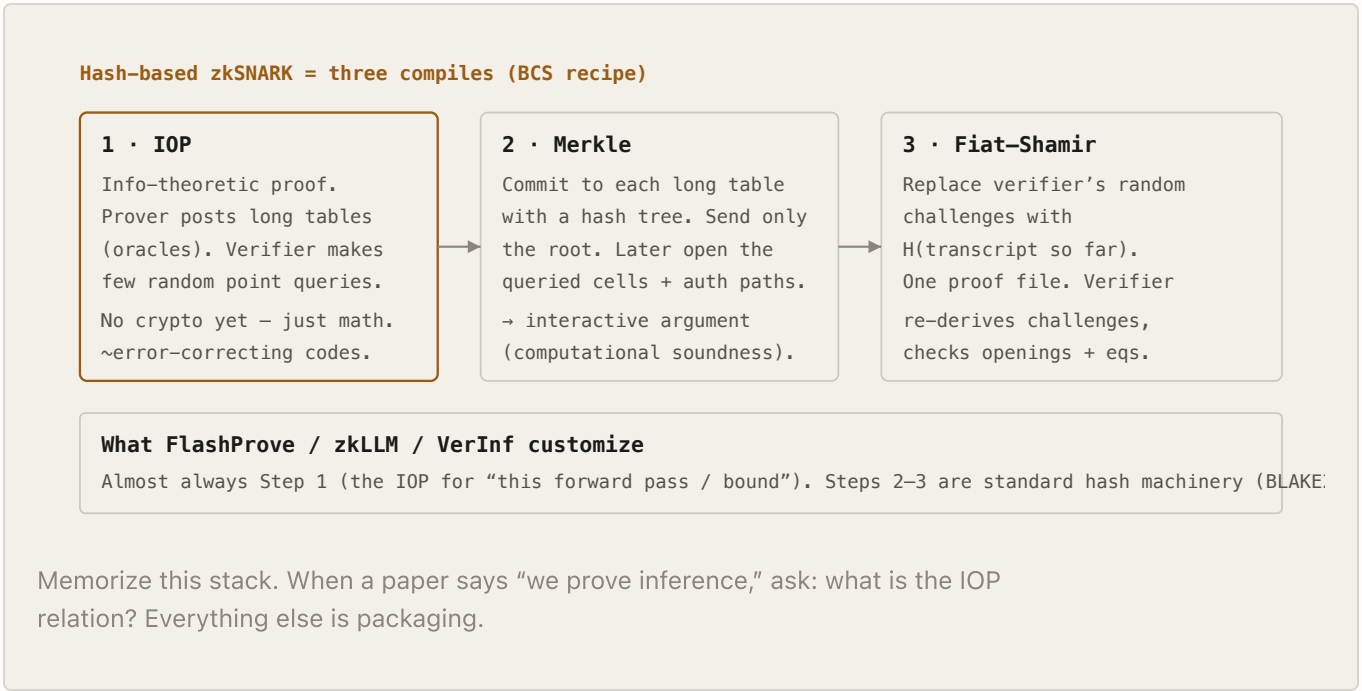
1. **Interactive Oracle Proof (IOP):** in a pretend world, the prover posts huge tables, and the verifier only looks at a few random cells. If the prover lied, those random peeks catch the lie with high probability. This layer is mostly information-theoretic math.
2. **Merkle trees:** in the real world you cannot email a terabyte table. So the prover publishes only a short fingerprint (the Merkle root). Later, when the verifier asks for cell number 17, the prover reveals that cell plus a short authentication path that matches the root.

3. **Fiat–Shamir:** in the real world you also cannot wait for a live verifier to send random challenges. So each "random" challenge is replaced by a hash of the conversation so far. The whole conversation becomes one proof file.

People sometimes call the finished object a **SNARK** (Succinct Non-interactive ARgument of Knowledge). After Merkle hashes enter, soundness is computational: a cheater with a magic hash-breaking computer might forge proofs. Cryptographers then say **argument** instead of information-theoretic **proof**.

When theorems pretend the hash is a perfect random function, that pretend world is the **random oracle model**. Real systems plug in BLAKE2 or SHA-256 and hope it is random enough.

Important: zkLLM (§6) does *not* use this sandwich for binding weights. It uses an algebraic polynomial commitment (Hyrax, §§15–16) instead of Merkle+RS for that job. Same wish-list; different commit crypto.



Memorize this stack. When a paper says "we prove inference," ask: what is the IOP relation? Everything else is packaging.

Words people mix up

Proof (info-theoretic)

Even infinite compute can't fake it (except tiny random chance).

IOP layer feels like this.

Argument

A cheater with a huge computer who can break the hash might fake it.

Merkle / SNARK layer.

Random oracle

Pretend hash H is a perfect random function. Theorems assume that;

BLAKE2 is the stand-in.

Practical English

FlashProve is an argument in the random-oracle model: "fake a proof" ≈ "break BLAKE2 / get absurdly lucky."

Lean theorems are about this idealized world. Real BLAKE2 is a belief, not a proof.

5 · Reed–Solomon encoding + random mask — "hide secrets while still allowing spot checks"

USED BY §7 FlashProve | hash-based backends | not Hyrax itself

Problem A (soundness): the verifier will only look at a few cells of a long table. If the table is just the raw secret numbers, a cheater could put correct values only in the cells that get asked, and garbage everywhere else.

Problem B (zero knowledge): if those few opened cells are literal pieces of the secret, the verifier learns the secret bit by bit.

What Reed–Solomon encoding does about Problem A: take your short secret message and turn it into a much longer row by treating the message as a polynomial and evaluating that polynomial at many points. The long row is called a **codeword**. Honest codewords for different messages disagree in most positions. So if a cheater changes lots of entries, almost any random peek hits a wrong value. That is the same idea as an error-correcting code on a CD: redundancy makes corruption obvious.

What the random mask does about Problem B: before encoding, the prover mixes in a few random coefficients. Then any small number of opened cells looks random, and those cells do not reveal the original secret. There is a budget: if you open more than about t cells on the *same* encoding, the randomness runs out and privacy can break. People call t the **query budget**.

What it is used for: committing to secret weights or secret intermediate values in a way that supports both "random spot checks catch cheating" and "spot checks do not leak the

secret." FlashProve's commit layer is this story. Hyrax solves the same job with group elements instead (§16).

LATER IN PAPERS

Commit-and-prove systems refresh the random mask (new encodings of the same secret) when the query budget is used up. That is an application detail, not the definition of Reed–Solomon.

Reed–Solomon + random mask = zero-knowledge encoding

Message → polynomial → codeword

weights $m_0 \ldots m_{k-1}$ + random $r_0 \ldots r_{t-1}$

$f(X) = r_0 + r_1 X + \ldots + r_{t-1} X^{t-1}$

$+ m_0 X^t + \ldots + m_{k-1} X^{t+k-1}$

Evaluate f at $n \gg k$ points → long row

Different messages → rows differ almost everywhere

Why ≤ t opens leak nothing

Any t values of f are fixed by the random mask alone — statistically independent of the weights m.

t = query budget. Open more than t columns ⇒ privacy can break.

FlashProve consequence

Commit once (anchor table). Each proof opens a few columns → budget depletes. So make many leaf tables with each linked to the anchor ("same weights, new randomness"). One commitment ≈ millions of proofs until leaves

College linear algebra: a degree-<d polynomial is fixed by d points. Random low coefficients make the first t samples look random.

6 · Unique decoding vs list decoding — "how wrong can a row be and still mean one message?"

USED BY §7 FlashProve | proximity tests in hash SNARKs

Problem: the verifier only sees a few opened cells, so the committed row might be noisy. You must decide how close to a perfect codeword is "close enough."

If you demand the row be extremely close to a codeword, there is usually only **one** possible message that fits. That is **unique decoding**. It is safer, but you typically need more opens, so proofs get slower.

If you allow a larger error, several different messages might still fit the opened cells. That is **list decoding**. It needs fewer opens (faster), but now you must pin down *which* message you meant. Systems often add an extra random evaluation called an **out-of-domain sample** (OOD) for that pinning step.

A **proximity test** is the protocol piece that asks: “is this committed row close enough to some codeword, and does it satisfy these linear claims?” Unique vs list decoding is about how large “close enough” is allowed to be. (Related name you will see in STARK literature: **FRI** — a different proximity engine for Reed–Solomon; FlashProve’s writeups lean on the RS+Merkle+OOD story above rather than teaching FRI as a named protocol.)



7 · Merkle trees — “commit to a huge table without mailing the huge table”

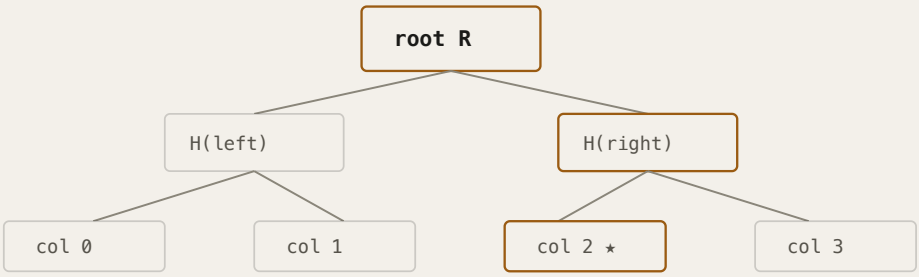
USED BY §7 FlashProve | §5 tap hashes (non-ZK cousin) | not Hyrax PCS

Problem: the IOP wants the verifier to open a few cells of a huge table. Sending the whole table destroys succinctness. Letting the prover just *tell* you cell 17 is dangerous, because the prover could change answers depending on which cell you ask.

What a Merkle tree does: hash every cell, then repeatedly hash pairs of hashes until one top hash remains. That top hash is the **Merkle root**. Publish the root early. Later, to open cell 17, send the cell value plus the sibling hashes along the path to the root. The verifier re-hashes upward. If the final hash matches the published root, that cell is authentic for the committed table.

What it is used for: turning “query an oracle table” into a short, checkable opening. This is the main bridge from the IOP math game to a real-world hash-based argument. Cankaya’s Plan A taps also hash-and-sign blobs — same “fingerprint” intuition, without a SNARK on top.

Merkle commitment: bind a long vector; open one cell later



Open col 2

Send value(col 2) + sibling hashes on the path to R. Verifier re-hashes upward; accepts iff root matches the Prover can't change an unopened column later without breaking the root (assuming H is collision-resistant).

IOP said "query oracle[i]". Merkle implements that query without shipping the whole oracle.

8 · Multilinear extension — "make a spreadsheet into a polynomial you can quiz"

USED BY §6 zkLLM | §7 FlashProve | any sumcheck-heavy system

Problem: your data is often a big table with addresses made of bits (row 0/1, column 0/1, ...). Sumcheck and friends want to talk about a *polynomial*, not a spreadsheet.

What we do: there is a standard, unique way to build a polynomial that matches the table on every 0/1 corner and is degree at most 1 in each variable. That polynomial is the **multilinear extension** of the table.

What it is used for: when a paper says "run sumcheck on P," P is often this extension of some big table of values from the circuit, not a random made-up polynomial. Hyrax also talks about committing to (multilinear) polynomials over the weights.

9 · Random batching — "check many claims with one random mix"

USED BY all ZK §§ | §6 | §7 | §8

Problem: after arithmetization you may have thousands of separate equations. Checking them one by one is still too expensive.

What we do: pick a random field element r and check a random weighted mix of the claims (for example $C_1 + r \cdot C_2 + r^2 \cdot C_3 + \ldots$). If every claim is true, the mix is true. If any claim is false,

the mix is a nonzero polynomial in r, so a random r almost never makes it accidentally true. That is Schwartz–Zippel again.

What it is used for: collapsing many checks into one check before sumcheck, a proximity test, or a batch of Hyrax openings.

Two bookkeeping tricks used constantly

Multilinear extension

A table on corners $\{0,1\}^n$ becomes ONE polynomial that matches the table on those corners, and is degree ≤ 1 in each variable.

Sumcheck often sums that polynomial.

Random batching

Many claims $C_1 \ldots C_k$ true?
Pick random r; check
$C_1 + r\,C_2 + r^2\,C_3 + \ldots = 0$.

If any C_i is false, the combo is a nonzero poly → random r catches it.

Again: Schwartz–Zippel

Batching is not a new idea — it's "one random point checks many equations at once."

When a paper says "we fold claims with a random challenge," this is what they mean.

10 · Sumcheck — "verify a gigantic sum without adding every term"

USED BY §6 zkLLM | §7 FlashProve (~2000 / request) | §8

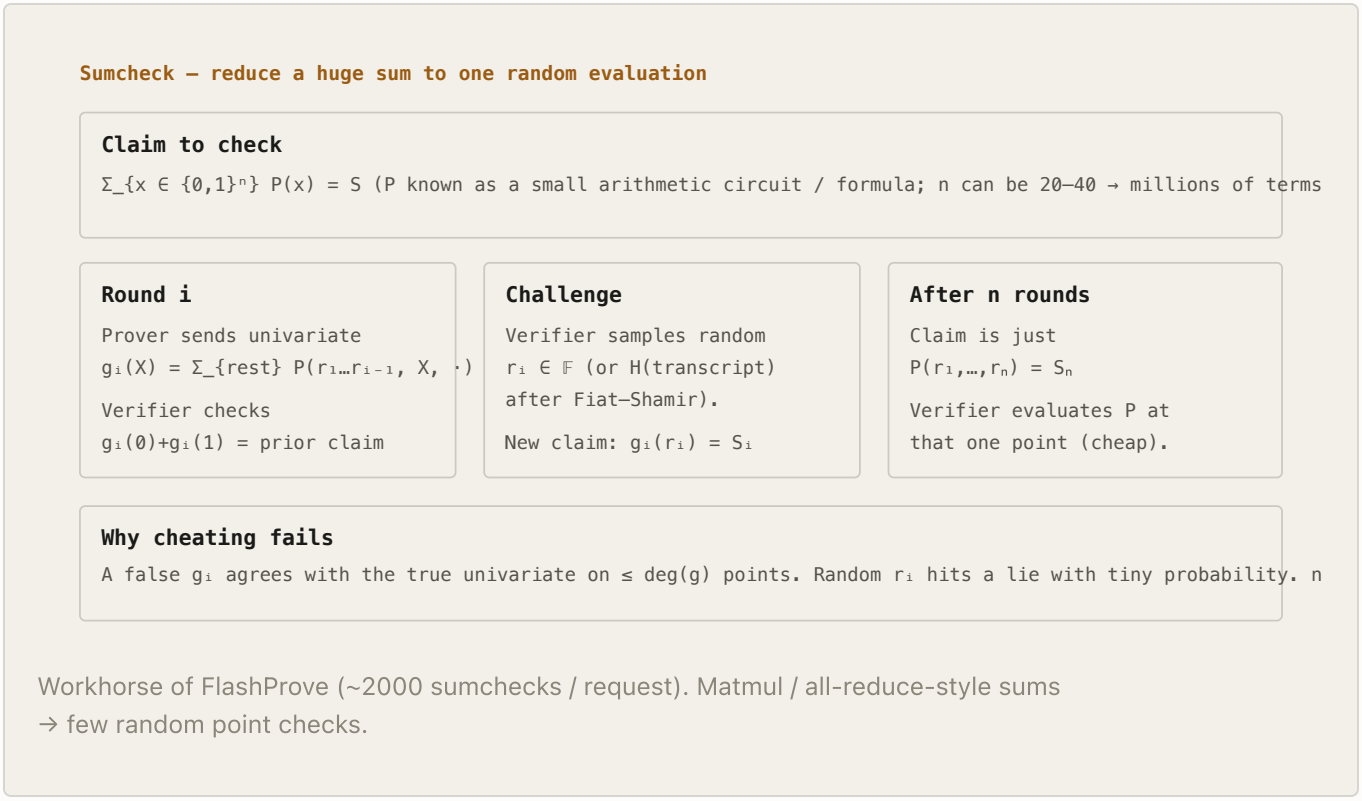
Problem: many circuit checks look like "the sum of this polynomial over millions of bit-strings equals S." Adding millions of terms is too slow for the verifier.

What sumcheck does: the prover and verifier play a short game, one variable at a time. In each round the prover sends a small one-variable polynomial that claims to summarize the remaining sum. The verifier checks that this small polynomial is consistent with the previous claim (plug in 0 and 1 and add). Then the verifier picks a random value for that variable, and the claim shrinks. After enough rounds, only one random evaluation of the big polynomial remains, and the verifier checks that single point.

Why cheating fails: if the prover sends a wrong small polynomial, it can agree with the truth at only a few points. A random challenge almost always lands off those points. Again: Schwartz–Zippel.

What it is used for: matrix multiplies and other huge structured sums inside ML circuits become sequences of sumchecks instead of brute-force addition. FlashProve is sumcheck-

heavy; zkLLM uses sumcheck on top of Hyrax openings.



Workhorse of FlashProve (~2000 sumchecks / request). Matmul / all-reduce-style sums → few random point checks.

11 · Lookups — "handle ugly functions without building a nightmare circuit"

USED BY §6 zkLLM (tlookup · zkAttn) §7 FlashProve range checks everywhere

Problem: neural nets use nonlinear functions that are painful to write as a few multiplications (exponentials, comparisons, clipped activations). Expanding them into huge multiplication circuits makes proofs enormous.

What a lookup does: publish a table of allowed pairs (input, output) ahead of time. The prover claims: every secret pair that appeared in my run is a row of that public table. There are standard protocols that check this claim using sumcheck-style algebra.

Catch: the public table must stay reasonably small, or the proof becomes huge. That is why quantized / clipped nonlinearities show up in zkML systems.

What it is used for: Softmax pieces, SiLU, range checks, comparisons — anything that is easier to tabulate than to arithmetize gate-by-gate. zkLLM names its gadgets **tlookup** (generic table membership via a rational identity) and **zkAttn** (Softmax shift + digit exp tables + a tolerance band on row sums — not exact log-sum-exp).

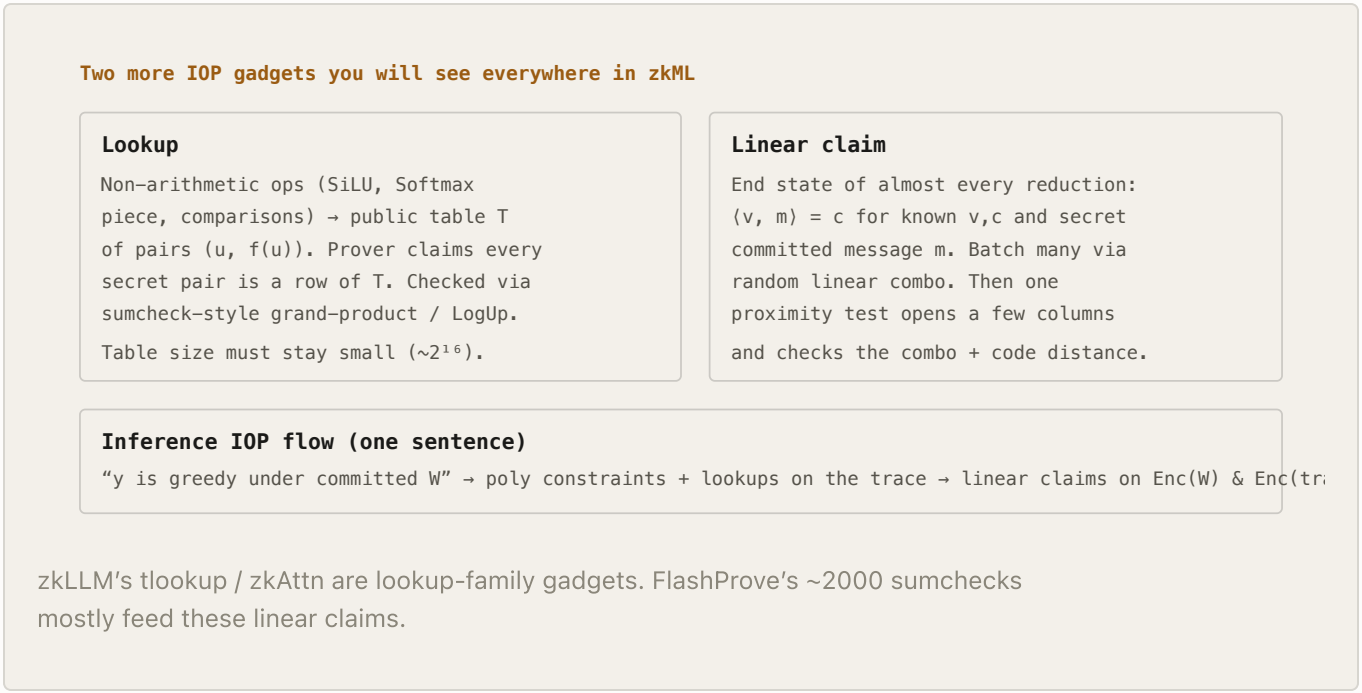
12 · Linear claims — "the common final shape of almost every check"

USED BY §7 FlashProve | hash proximity finishers

Problem: after sumchecks and lookups, you still need one simple kind of statement the proximity test knows how to finish.

What a linear claim is: "for this secret committed vector m, and this public vector v that we both know, the weighted sum ⟨v, m⟩ equals a public number c."

What it is used for: nearly every high-level check in the hash stack eventually reduces to a handful of linear claims. Then you batch them and feed them into a proximity test that opens a few Merkle columns. In the Hyrax stack, the analogous "finisher" is often ProveEval at random points rather than Merkle column opens — same reduction spirit, different opening API.



13 · Fiat–Shamir — "turn a live conversation into a file you can email"

USED BY all non-interactive ZK §§ | §6 | §7 | §8 | §10

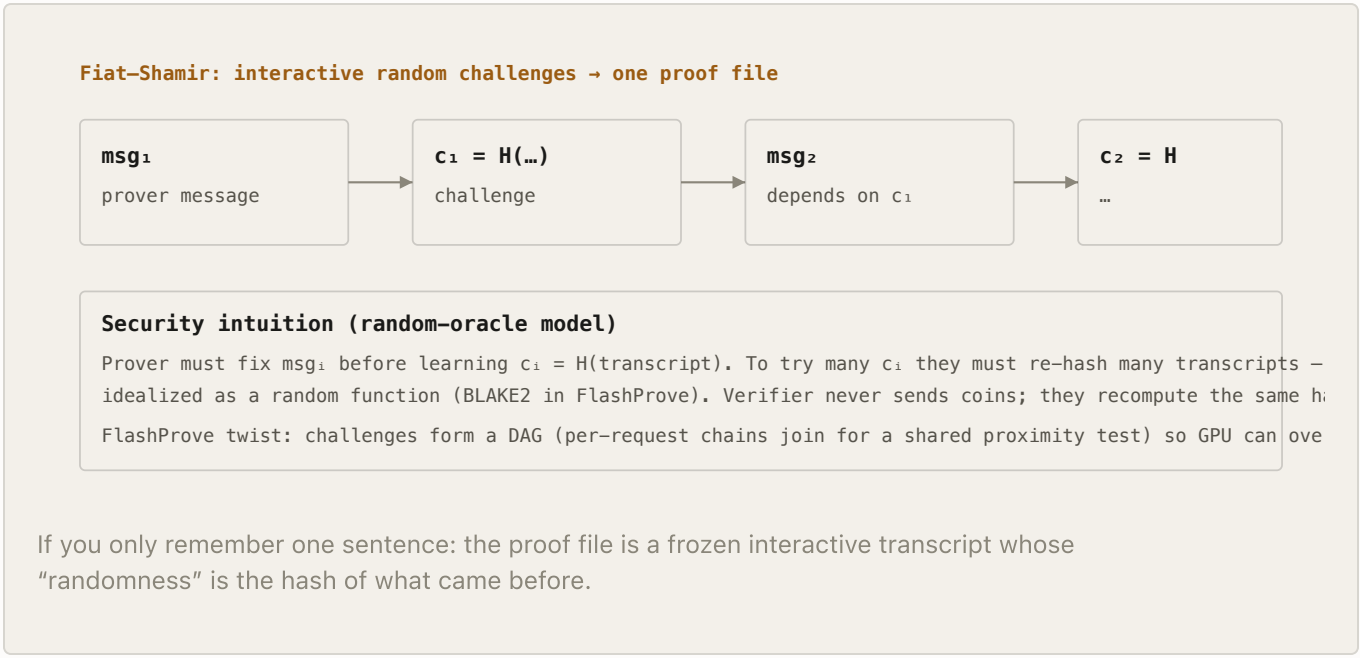
Problem: the interactive protocol needs the verifier to keep sending fresh random challenges. In practice you want a single proof file that anyone can check offline.

What Fiat–Shamir does: replace each verifier challenge with a hash of everything that has been said so far:

```
next challenge = Hash(public parameters ‖ previous messages)
```

The prover writes the whole transcript into the proof. The verifier recomputes the same challenges from the hashes and checks the openings and equations. Because the challenge is determined by prior messages, the prover cannot first see the challenge and then invent a matching lie (unless the hash is broken).

What it is used for: making interactive protocols into non-interactive SNARKs / STARKs. FlashProve's twist: challenges form a DAG (per-request chains join for a shared proximity test) so GPU work can overlap — still Fiat–Shamir, just with a dependency graph.



If you only remember one sentence: the proof file is a frozen interactive transcript whose "randomness" is the hash of what came before.

14 · Commit-and-prove — "nail the secret once, then prove many public claims about it"

USED BY §6 zkLLM §7 FlashProve §8 VerInf

Problem: sometimes the secret (for example a model's weights) is reused across many statements. You want to bind that secret once, then prove many different public claims against the same binding, without re-sending the secret.

What commit-and-prove means: first publish a commitment C. Later, for each new public claim, produce a proof that refers back to that same C. The commitment is binding: you should not be able to open it to two different secrets. Separately, encodings may need fresh randomness so zero knowledge does not wear out after many openings.

How C is built differs by stack: FlashProve → Merkle root of RS+mask encodings. zkLLM → Hyrax group commitment to the weight polynomial. VerInf → same commit-and-prove pattern, with a different public statement (Û).

What it is used for: “I committed to one model; here are many inference receipts against that same model.”

15 · Discrete log & elliptic-curve groups — “why Hyrax can be short”

USED BY §6 zkLLM (Hyrax) | algebraic PCS family | not FlashProve’s commit layer

Problem: Merkle commitments are easy to understand, but every opening costs path hashes, and proof size grows with how many cells you open. People wanted a different deal: a tiny commitment object, and a way to prove “the committed polynomial evaluates to y at this random point” without shipping half the table.

What a group gives you: think of public points G, H, … on an elliptic curve as “atoms” you can add and scale, but you cannot efficiently reverse “how many times did I add G to get this point?” That hard reverse is the **discrete-logarithm** problem. A **Pedersen-style commitment** looks like C = a·G + r·H: it binds the secret a (you cannot open C to two different a’s without breaking discrete log), and the random r hides a (so C alone does not leak a).

What you need to remember (not a crypto course): hash assumptions (“break BLAKE2”) and discrete-log assumptions (“solve DL on this curve”) are different hardness stories. FlashProve’s commit layer bets on hashes. zkLLM’s Hyrax bets on discrete log in an elliptic-curve group. You can believe one and still be nervous about the other.

Trusted setup vs transparent: some algebraic SNARKs need a special ceremony to make public parameters (toxic waste if leaked). Hyrax as used in zkLLM is in the **transparent** family — no ceremony with secret leftovers — which is why the deck’s zkLLM table says “no trusted setup.”

16 · Polynomial commitment schemes & Hyrax — “bind a poly, open at random points”

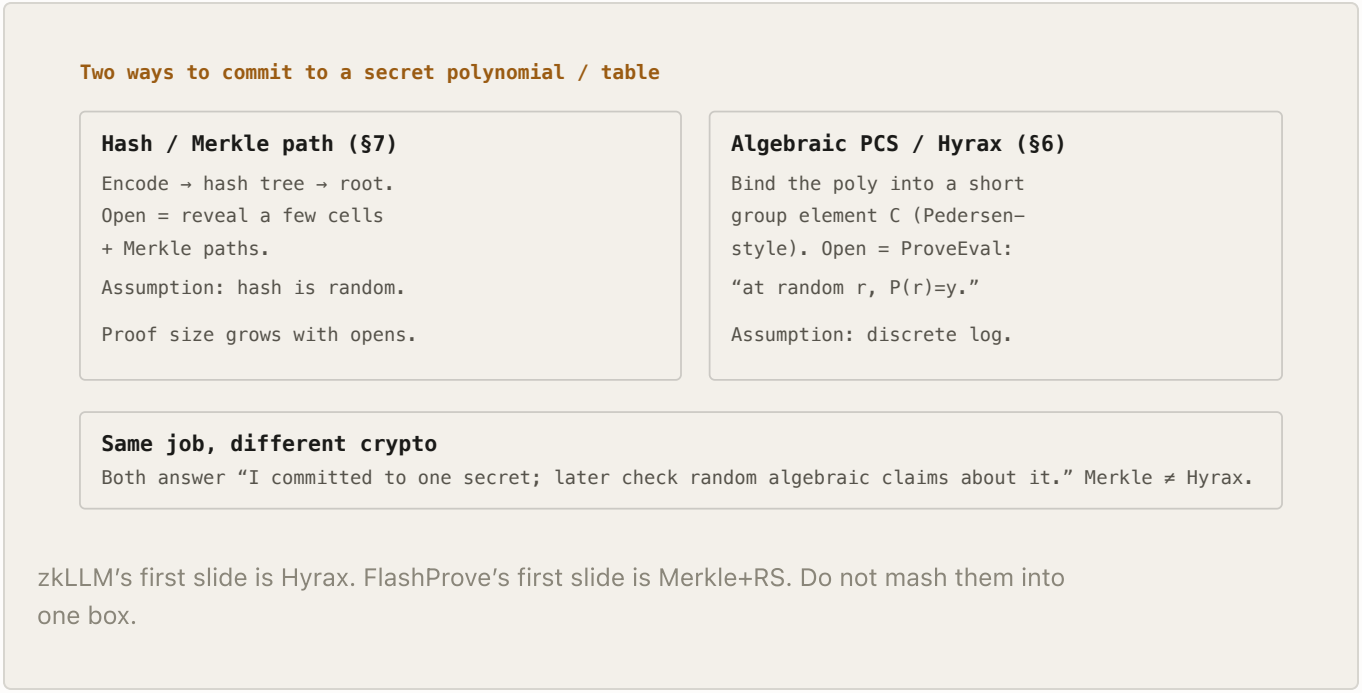
USED BY §6 zkLLM | any algebraic-PCS zkML

Problem: after arithmetization and sumcheck, the verifier often needs to believe “this huge secret polynomial P, evaluated at a random point r, equals y” — without seeing all of P.

What a PCS is: a **polynomial commitment scheme** is an API with two moves. (1) *Commit:* turn P into a short object C. (2) *Open / ProveEval:* later prove that P(r) = y for public r, y, relative to that same C. Binding means you cannot open C to two different polynomials. Hiding (when required) means C alone does not leak P.

Hyrax in one paragraph: Hyrax is a transparent PCS built from Pedersen-style commitments and an inner-product / bulletproofs-flavored argument (people say IPA for inner-product argument). zkLLM commits the weights W into C with Hyrax, then uses ProveEval at Schwartz–Zippel-random points as the finisher after sumchecks and lookups. Tiny proofs (~hundreds of bytes in the published 13B setting) come from this algebraic opening style, not from Merkle paths.

Names you may meet later (not needed for this deck, but so they do not scare you): KZG (pairing-based PCS, usually needs a setup), IPA/Bulletproofs (pairing-free, proof size grows slowly with degree), FRI (hash-based proximity for Reed–Solomon — lives next to stack A, not Hyrax).



17 · Choosing the public statement — “exact · clipped-gap · unexplained Û”

USED BY §6 exact circuit | §7 greedy / clipped-gap | §8 Û bound | §5 DiFR contrast

Problem: cryptography proves whatever statement you feed it. If you ask the wrong question, you get a perfect proof of the wrong policy claim. Float nondeterminism and steganography make “bit-exact same tokens as the GPU” a brutal statement.

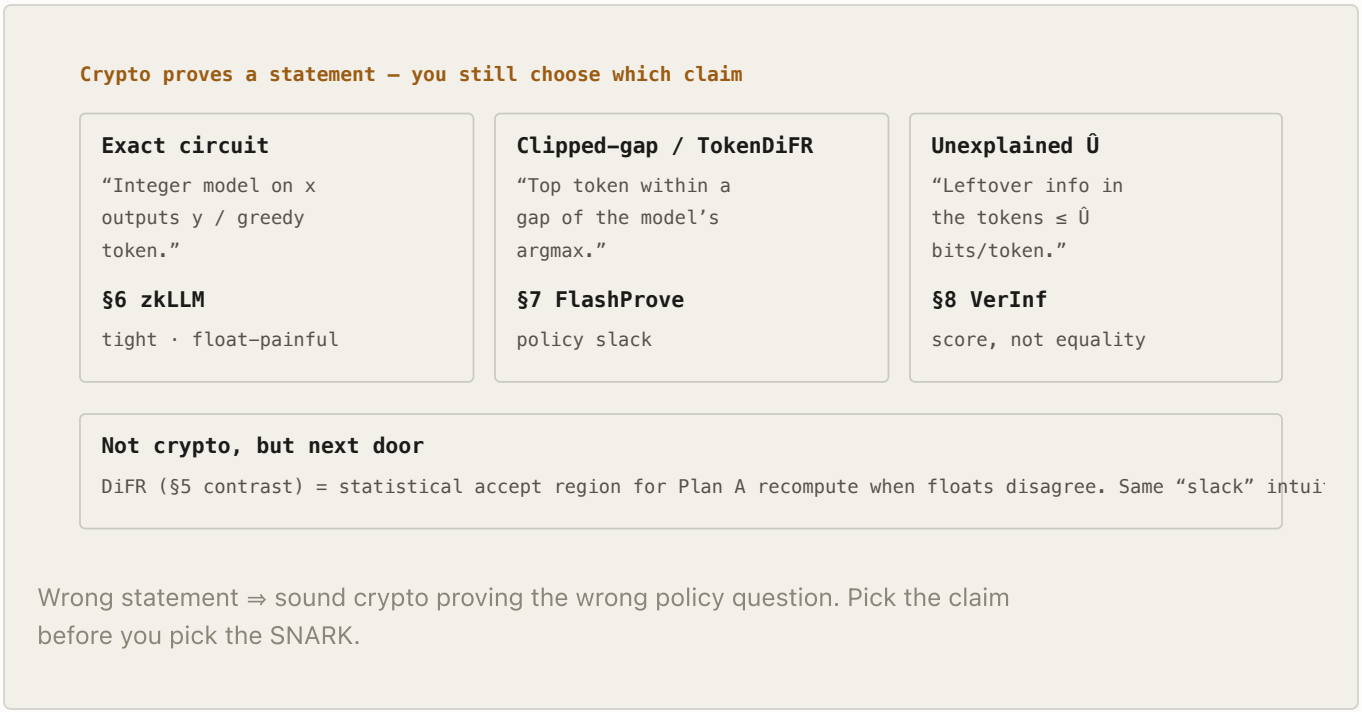
Three statements this deck actually uses:

- **Exact integer circuit** (§6 zkLLM): the committed quantized model on public (or committed) input x produces output y. Strongest “receipt.” Softmax is only proven up to

zkAttn's tolerance band.

- **Greedy / clipped-gap** (§7 FlashProve): either exact-match greedy of the integer model, or a **clipped-gap / TokenDiFR-family** policy — the emitted token is allowed if it sits inside a gap of the model's preferred choice. Slack is a *policy* choice, not a new SNARK.
- **Unexplained-info bound Û** (§8 VerInf): the committed model explains a (possibly hidden) token stream well enough that leftover unexplained information is ≤ a public Û bits/token. This is an information-theoretic *score*, not bit-exact equality. Wrong model / stego / garbage → Û blows up. Threshold Û is a policy knob.

DiFR next door (not a SNARK): Plan A recompute under float chaos often needs a statistical accept region (DiFR). Same "budgeted slack" intuition as clipped-gap / Û, but the engine is plaintext recompute, not a proof.



When you open a paper, ask first: *what is the statement?* Then ask which stack (hash PCS vs Hyrax vs zkVM) proves it.

18 · zkVM, recursion, folding — "prove a program ran" (§10)

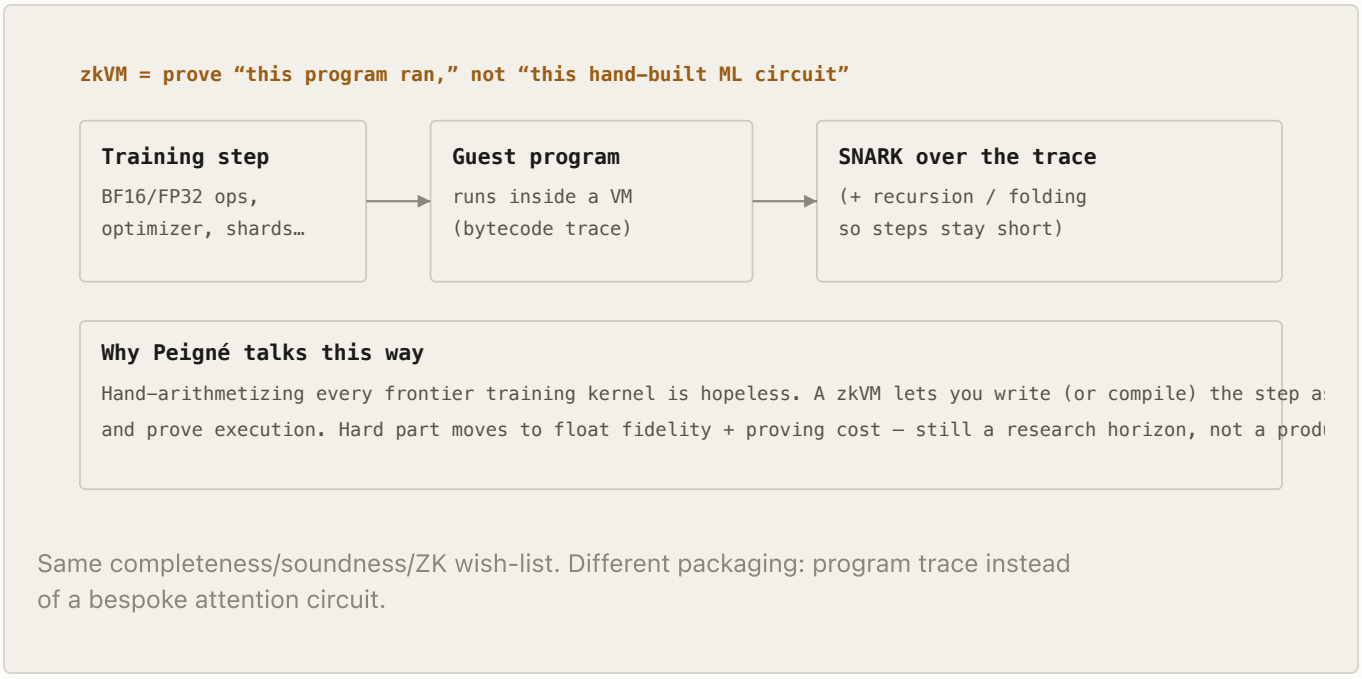
USED BY §10 Peigné | training-ZK horizon

Problem: hand-building an attention/matmul circuit for every frontier training kernel (BF16/FP32 optimizer steps, sharding, collectives) does not scale as a research program. You want to write the step as a *program* and prove that the program executed.

What a zkVM is: a virtual machine whose every step obeys public transition rules. The prover produces a long **execution trace** (registers, memory, flags at each clock). Arithmetization says "every consecutive pair of rows obeys the VM rules, and the public I/O matches the claim." Then a SNARK (often hash-based or folding-based under the hood) proves that statement. You prove "this guest program ran," not "this one-off Softmax gadget."

Why recursion / folding show up: a training step's trace is enormous. **Recursion** means a SNARK that verifies other SNARKs, so you can compress many chunks into one short proof. **Folding** (Nova-style intuition): incrementally combine two instance-witness pairs into one, paying a small cost per step instead of re-proving the world from scratch. You do not need the Nova/Halo2 paper trail to read Peigné — you need the idea "long computation → many small proven chunks → one short proof."

What Peigné is arguing: in principle, BF16/FP32-aware zkVM proofs of frontier training steps are a coherent research target. There is no end-to-end prototype in this briefing. Treat §10 as the training-side horizon that uses stack C, while §§6–8 are inference Plan B engines.



19 · How the tools fit together (three recipes)

MAP §7 ≈ recipe A §6 ≈ recipe B §10 ≈ recipe C

§8 = pick statement + A or B backend

```
Shared floor (everyone):
  fields · Schwartz–Zippel · arithmetization · Fiat–Shamir · pick a statement (§17)

Recipe A — hash / IOP  (§7 FlashProve; many VerInf-shaped backends):
  1. Equations + lookups
  2. Reed–Solomon + random mask
  3. Merkle commit
  4. Sumcheck / lookups / batching → linear claims
  5. Proximity test (Merkle opens; unique/list + OOD)
  6. Fiat–Shamir → one proof file

Recipe B — algebraic PCS  (§6 zkLLM):
  1. Equations + lookups (tlookup / zkAttn)
  2. Hyrax commit to weight polynomial → C
  3. Sumcheck / lookups / batching
  4. Hyrax ProveEval at random points (discrete-log assumption)
  5. Fiat–Shamir → one (often tiny) proof file

Recipe C — zkVM  (§10 Peigné horizon):
  1. Compile / write the training step as a guest program
  2. Arithmetize the VM trace (floats are the hard fidelity problem)
  3. Prove chunks; recurse / fold into a short proof
  4. Fiat–Shamir → one proof file
```

Concrete systems in §§6–8 and §10 are applications of these recipes, not replacements for this section. If a later section name-drops a tool you forgot, jump back here — each subsection's *Used by* line tells you whether that paper needs it.

> STUDY CHECKLIST
>
> For each tool, say out loud: (a) what everyday problem it solves, (b) which deck direction uses it, and (c) what would go wrong if we deleted it. Tools: finite field, Schwartz–Zippel, arithmetization, IOP sandwich, Merkle, Reed–Solomon+mask, unique vs list decoding, multilinear extension, batching, sumcheck, lookup, linear claim, Fiat–Shamir, commit-and-prove, discrete log / Pedersen, PCS / Hyrax, statement choice (exact · gap · Û), zkVM / folding. If you cannot answer (a) in a full sentence, re-read only that subsection.

> PRIMARY REFS
>
> Schwartz–Zippel (watch first): YouTube wNghCHYR6Vo · Textbook overview: Chiesa–Yogev *Proofs, Arguments, and Zero-Knowledge* · IOP compilation: Ben-Sasson–Chiesa–Spooner · Hyrax PCS: Wahby et al. · Then read a concrete system as an application: §6

> zkLLM (recipe B) · §7 FlashProve (recipe A) · §8 VerInf (statement §17) · §10 Peigné (recipe C)

SURVEY · TREATY FRAMING

1. Wasil et al. — verification method menu

arXiv:2408.16074 · 2024

Who

Wasil, Reed, Miller, Barnett et al. — survey aimed at international AI agreements (access assumptions + evasion sketches).

Maturity

Survey / coordinate system. Not a deployable stack. Useful to prevent "we solved verification" talk after reading one mechanism.

Problem it frames: If states agree "don't secretly train too big / don't secretly build too many DCs," what instruments exist? Two violation types: **unauthorized training** and **unauthorized datacenters**.

Lineage: Entry point for the rest of this deck. Later works sit in its hardware-dependent / access-dependent cells (Shavit, FlexHEG, location proofs, workload monitoring).



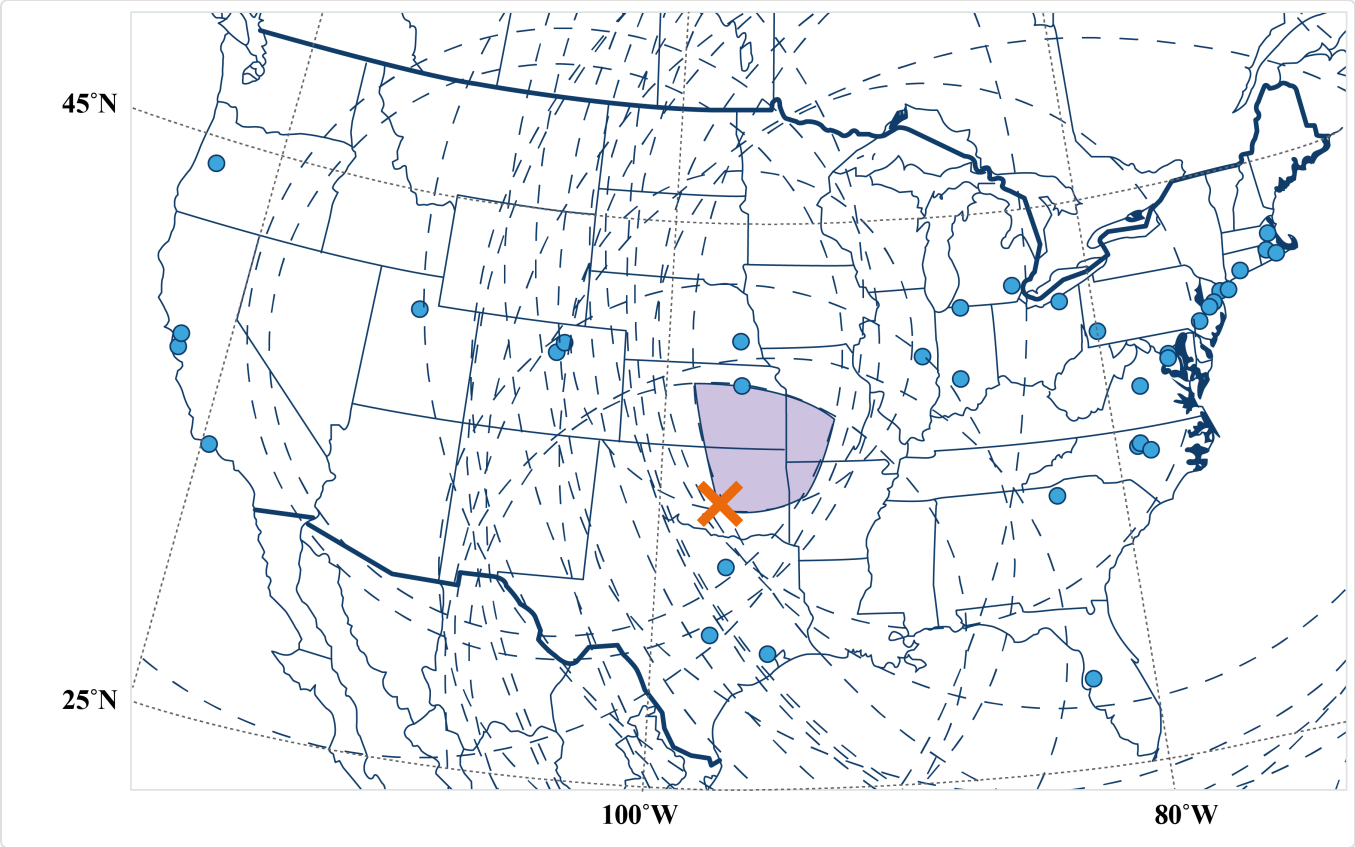
Wasil taxonomy: cooperation cost ↑ left→right; technical bite ↑ left→right. Not a system design — a menu.

What to remember technically

- Methods differ in **access assumptions**: unilateral national technical means vs negotiated entry vs pre-agreed chip behavior.

- Hardware-dependent methods are highest bite and hardest diplomacy.
- Chip location ≠ training proof. Location answers “where is the silicon?”; workload methods answer “what did it run?”

One Wasil cell — chip location proofs — looks like this: you are not proving “this GPU trained legally.” You are proving (approximately) “this registered chip is physically near claimed site X,” so a dark training cluster in an undeclared country is harder to hide. Delay-based / multi-anchor checks exploit physics: light in fiber has a max speed, so round-trip time to a trusted probe sets a distance upper bound.



How to read the figure. Blue dots = trusted *anchors* (known-location probes / PoPs). Dashed arcs = “this chip answered me in ≤ T ms ⇒ it cannot be farther than ~c·T/2 in fiber.” Purple blob = intersection of several such bounds (where the chip is allowed to sit). Orange X = true location (ideally inside the blob). Spoofing “I’m in Virginia” while sitting in another country fails if enough distant anchors force the intersection away from Virginia. This is only a *where* check — still need workload methods (Shavit / joshc / FlexHEG / …) for *what ran*.

CLASSICAL COMPUTE MONITORING

2. Shavit — Catch a Chinchilla (+ Proof-of-Learning)

arXiv:2303.11341 · 2023 · PoL primitive: Jia et al. 2021

Who

Yonadav Shavit — early blueprint for monitoring large training runs via chip-level weight fingerprints + training transcripts + supply-chain coverage.

Maturity

Blueprint. Problem decomposition, not an industry-deployed end-to-end system as of 2026.

One sentence: force frontier training onto registered ML chips that leave sparse *weight fingerprints*; require a transcript that explains those fingerprints (Proof-of-Learning-style checks); audit the supply chain so dark chips can't skip the net.

Lineage: Classical "what Claim A monitoring could look like." Gaps it leaves → later work: train↔infer ambiguity (Rahman); cryptographic transcripts (Peigné); active hardware caps instead of after-the-fact snapshots (FlexHEG).



PoL check (Jia et al. intuition)

Claim: W_T came from (W_0, D, A) under claimed optimizer steps

Verifier samples steps i → recompute / check consistency with snapshots

Mismatch ⇒ lying about process · Match + illegal FLOP ⇒ rule break

Refuse audit ≈ adverse inference (audit analogy)

Shavit stack: fingerprints on chip + proof the transcript explains them + no dark chips outside the registry.

Threat model that matters

Prover is a **covert adversary**: cheats only when it can look compliant. Design goal = make "look compliant while cheating" nearly impossible under random chip audits.

On chip

Firmware occasionally hashes / records HBM weight shards. Needs signed firmware + later physical spot checks (remote attestation alone is weak against a physical adversary).

In the DC

Transcript = data references, hparams, checkpoints, optimizer trajectory. Verifier samples steps; inconsistency or illegal scale ⇒ fail.

Open issues Shavit already flags

- Train vs infer lookalike occupancy of HBM.
- Legacy chips without secure firmware — small/medium training may escape.
- Vendor–prover collusion on backdoored firmware.

EMPIRICAL THIN LAYER

3. Rahman & Tajdari — train vs infer from NVML

arXiv:2606.19262 · TAIGR @ ICML 2026

Who

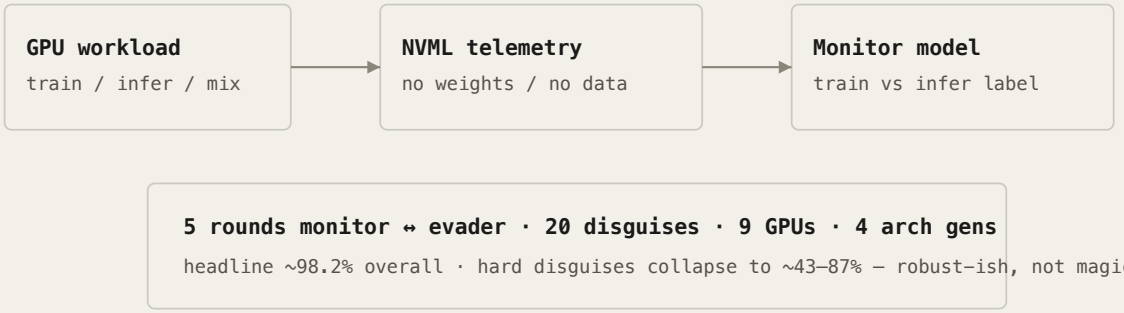
Robi Rahman (MIRI Technical Governance; ex-Epoch) and Sabiha Tajdari (UVA) — SPAR-originated empirical red-team of GPU workload classifiers.

Maturity

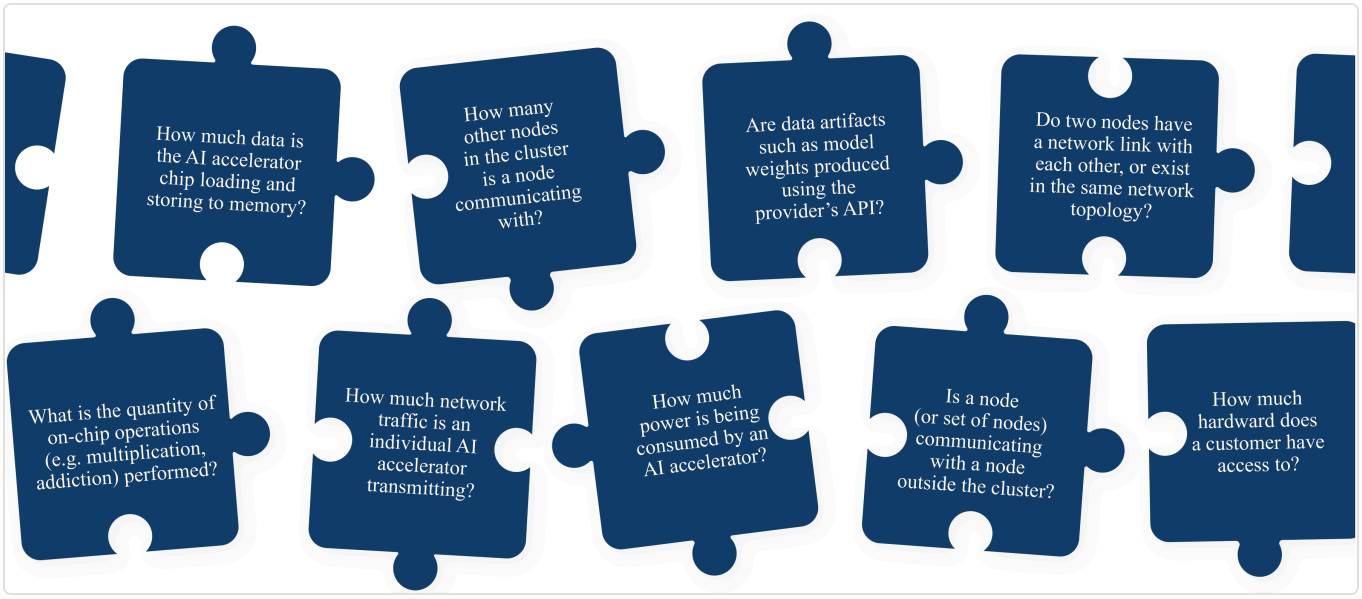
Empirical paper + public tooling lineage. Real numbers under adversarial disguise. Solves only a *thin slice* of Claim A (label train vs infer), not full rule verification.

Question: can you tell training vs inference from GPU telemetry (NVML) without touching weights or data?

Lineage: Attacks the gap Shavit flagged and that FlexHEG/joshc classifiers often assume. Complements interconnect-based detectors (communication volume) — different observation channel.



Thin layer only: "is this training?" not "did this training violate a FLOP license."
Empirically grounded where Shavit/ZK are still blueprints.



Example metric space for workload monitoring. Rahman asks whether this works under disguise.

Numbers to keep (and not overclaim)

- ~98.2% overall accuracy across their matrix.
- Under the hardest disguises, ~43–87% — robust overall, meaningfully degraded under sophisticated evasion; not "solved."
- Method: 5-round monitor–evader loop, multi-GPU / multi-arch — the right evaluation bar for any such detector.

NEAR-TERM SLOWDOWNS

4. joshc — TAPs, wipes, and covert RL

LessWrong / Alignment Forum · 2026

Who

joshc (Alignment Forum / LessWrong) — red-team essay on whether *fast, cheap facility mods* can slow training before treaty-grade hardware exists.

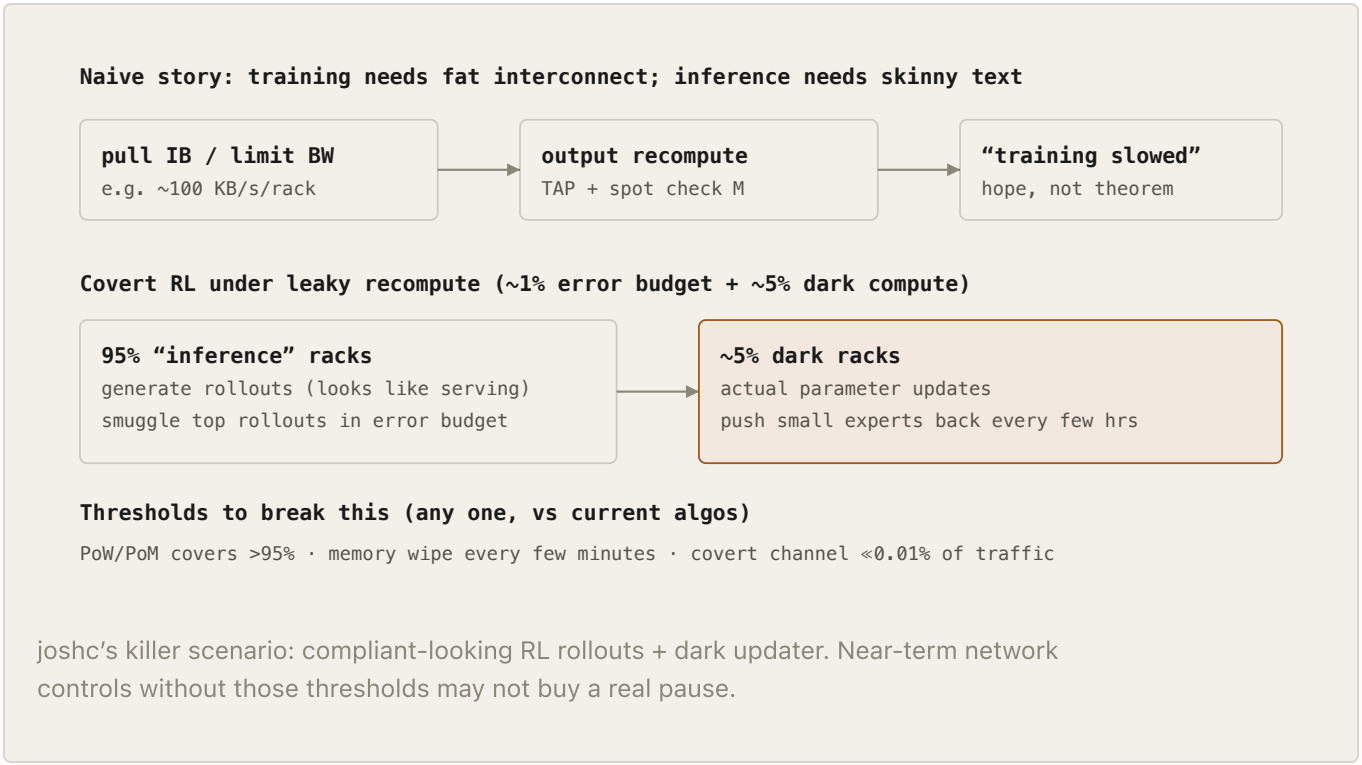
Maturity

Red-team / requirements doc. Not a product. Sets pass/fail bars for prototypes (TAP + recompute + wipe + accounting).

Question: before global FlexHEG-class silicon exists, can governments slow training by pulling InfiniBand, capping rack egress (~100 KB/s), TAP+recomputing outputs, wiping memory, demanding PoW/PoM?

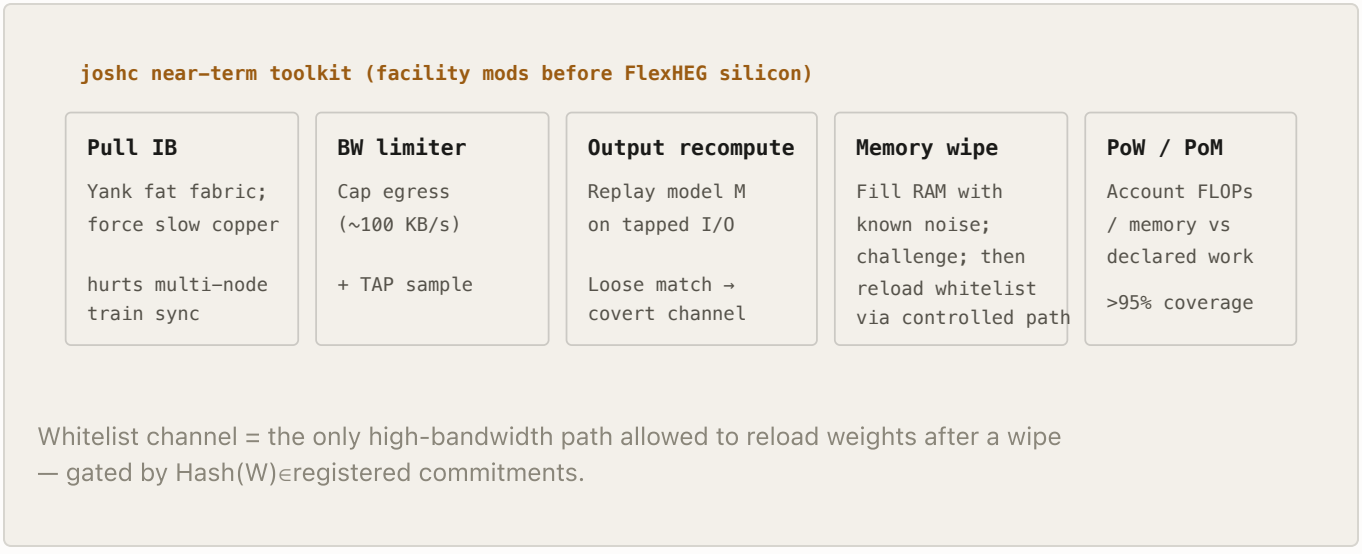
Naive lever: training is communication-hungry; inference is text-skinny. joshc's point: under *today's prototype strength*, that lever may fail against modern RL pipelines (rollouts look like inference; updates can hide in a small dark fraction of compute + leaky recompute slack).

Lineage: Stress-tests the "medium" rung (after you've seen how thin NVML train↔infer is). Motivates tighter recompute (Cankaya bit-exact / DiFR), better taps, and why FlexHEG remains the longer-term enforcement story.



joshc's killer scenario: compliant-looking RL rollouts + dark updater. Near-term network controls without those thresholds may not buy a real pause.

Facility toolkit (where each tool sits)



Whitelist channel = the only high-bandwidth path allowed to reload weights after a wipe — gated by Hash(W)∈registered commitments.

Five near-term tools (compressed)

TOOL	IDEA	AUTHOR'S READ
Pull IB	Replace ~400 GB/s fabrics with slow copper	~1y prep plausible
BW limiter + TAP	Cap egress; sample traffic	to ~100 KB/s plausible
Output recompute	Replay declared model M on tapped I/O	~1% nondeterminism budget = covert channel

TOOL	IDEA	AUTHOR'S READ
`Memory wipe`	Force verifiable fill; reload whitelist model	needs ≫99.99% coverage — hard
`PoW / PoM`	Account for FLOPs/memory spent on declared serving	useful if >95% coverage

> PROTOTYPE BAR
>
> If a demo doesn't close the RL-smuggling channel under realistic slack, it has not purchased a credible pause — regardless of how good benign accuracy looks.

NEAR-TERM RETROFIT · PLAN A

5. Cankaya — taps, challenges, and precise replay (Plan A)

System overview LW · taps Substack Apr 2026 · bit-exact 2606.00279 · DiFR (statistical slack) 2511.20621

Who

Naci Cankaya (MIRI technical governance) — whole-facility retrofit blueprint: capture I/O → random challenge → evaluate. Bit-exact paper is the Plan A numerics companion; taps note is the capture companion.

Maturity

Overview: **blueprint**. Taps: **feasibility + commodity-tech argument** (NS ready-ish; EW needs pilots). Bit-exact: **empirical + emulator prototype** for inference — not a training verifier; not yet a fielded AE standard.

Shared audit question: was this sampled request run on the whitelist model?

Plan A (near-term default): air-gapped audit environment (AE) holds the whitelist weights and *recomputes* the challenged forward pass in plaintext, then compares to tapped I/O. For that comparison to be a pass/fail (not a ~1% stego channel à la joshc), you need **precise replay** — ideally bit-exact under recorded metadata, or a disciplined statistical alternative (DiFR).

Plan B (later sections): replace AE recompute with ZK so the peer needn't see weights — zkLLM (§6), FlashProve (§7), or VerInf (§8). Same challenge loop; different evaluate engine.

Lineage: Turns joshc's TAP/recompute sketches into an arms-control-shaped architecture. FlexHEG is the longer hardware track; Peigné (§10) is the training-ZK cousin.

Properties checklist (before the diagrams)

PROPERTY	PLAIN ENGLISH	PLAN A (AE PLAINTEXT)	PLAN B (ZK)
Completeness	Honest clean prover passes	Needs bit-exact / DiFR so benign noise ≠ reject	Honest π verifies
Soundness	Cheater fails	Recompute mismatch / wrong W vs C	Bad witness can't forge π (or Û bound)

PROPERTY	PLAIN ENGLISH	PLAN A (AE PLAINTEXT)	PLAN B (ZK)
Confidentiality	Secrets stay secret	**Weak** — W and challenged I/O enter AE	**Stronger** — witness not revealed
Assurance	Field catch probability	Soundness × sample rate × tap coverage × stego slack	Same multipliers; π ≠ free lunch

Why plaintext AE recompute is not “the endgame”

1 · IP / data

Whitelist W and challenged x,y enter the AE in clear. Rival states / labs hate this.

→ confidentiality failure

2 · Mutual distrust

Whose GPU is the AE? Prover won’t trust verifier’s box; verifier won’t trust prover’s box either.

3 · AE is still a TCB

Air-gap ≠ zero leak. Need diodes, cross-checks, physical monitoring — Cankaya’s hard part.

So why is Plan A still the near-term default?

Because it is fieldable without frontier-scale ZK: if you can stomach AE confidentiality risk + build bit-ex you get real assurance against “swap the model on the wire.” Plan B (zkLLM / VerInf) replaces the AE evaluat confidentiality is the dealbreaker — it does not remove taps, sampling, or the need for soundness of the sta

Plaintext recompute answers soundness-ish checks by literally redoing the math. ZK answers them with a proof. Same audit question; different confidentiality cost.

Why plaintext recompute is not enough (even when it “works”): it can deliver completeness/soundness/assurance under good engineering, but it fails the low-trust confidentiality goal — rival labs/states will not ship frontier weights into the other party’s AE, and neither trusts the other’s silicon inside the cage. That is the explicit motivation for Plan B. Plan A remains the near-term default because ZK at frontier scale is still expensive / immature — not because confidentiality doesn’t matter.

Same audit question: was this sampled request run on the whitelist model?



Dead-simple memory aid

Plan A: I watch you recompute.

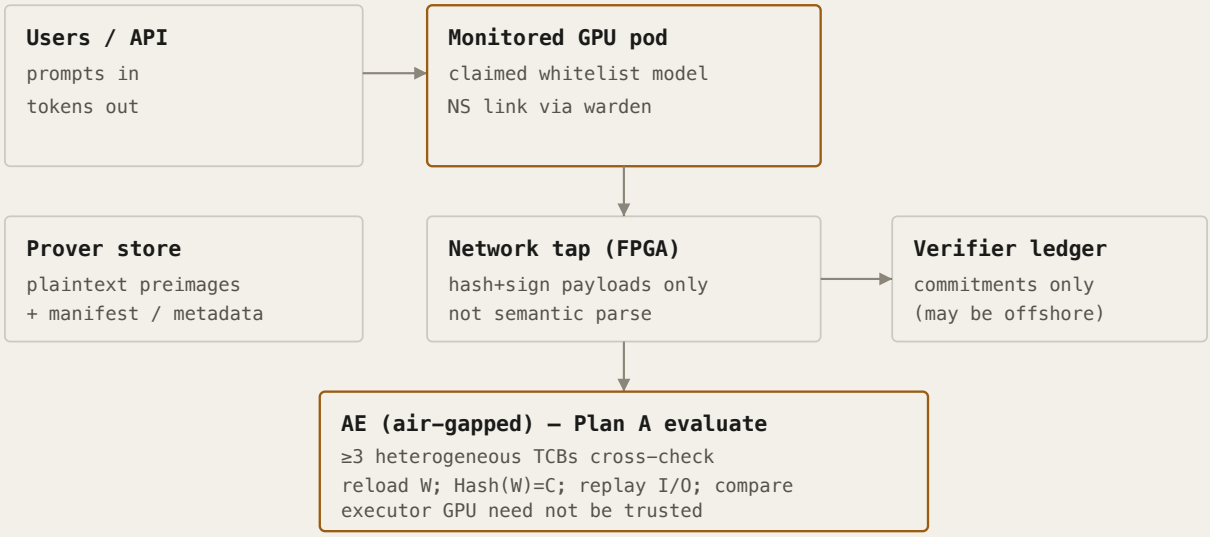
zkLLM: you prove the circuit executed (receipt).

VerInf: you prove the committed model explains the tokens; leftover ≤ Û (score).

Near-term default in Cankaya: Plan A. zkLLM = early Plan-B instance. VerInf = frontier-scale Plan-B core.

Inference side of Claim A / arms-control-like monitoring. Complements Peigné's training ZK; sits under Cankaya's tap → challenge → evaluate loop.

Cankaya Plan A sketch — taps + air-gapped audit environment (AE)



Adapted from Cankaya's system overview: online evidence capture is continuous; evaluation is random challenges. Plan B swaps the AE plaintext recompute for a ZK proof.

Cankaya's four doors on a challenged forward pass

1 · Weights	2 · I/O evidence	3 · Recompute	4 · Semantics
Hash(W) = C whitelist model?	preimage matches tap commitment	run W on input; output matches?	blacklist use? (CBRN, etc.)
Hard check	Hard check	Hard (bit-exact ideal)	Soft / inspectors

Doors 1–3 can all pass while door 4 fails: whitelist model doing banned things. Soft classifiers are not a substitute for 1–3.

Network taps (where commitments come from)

Where to put network taps (Cankaya Apr 2026 feasibility note)

North–South (frontend)	East–West (backend)
DC ↔ outside world (API, SSH)	GPU↔GPU · storage · all-reduce
cheap · active warden OK	optical split + random sample
cost ≪ 0.01% of facility TCO	~0.2–1.5% TCO (level-dep.)
do this first	copper scale-up (NVL*) stays blind

Tap outputs commitments (hash + timestamp + sign), not raw secrets to the peer state.

Prover can passively re-observe tap egress → near-term no mutually trusted silicon.

Storage fabric next (~0.3–0.5%): which weights loaded, datasets read, checkpoints written.

Biggest unknown: residual covert bps if only NS is tapped (Fermi ~10^2 bps – needs empirics).

Evidence capture layer under Cankaya's blueprint. Source: Cankaya, secure network taps (Apr 2026).

- **NS first:** frontend traffic is ~Gb/s-class in the note's Fermi — commodity FPGA / mid-range NIC; active wardens fit.
- **EW:** full mirror is infeasible; wide optical split + random sampling; need tensor metadata or bytes are meaningless.
- **Copper NVL*:** passive optical taps miss intra-rack scale-up — security falls back onto NS + storage.

Bit-exact inference verification (precise replay)

This is the load-bearing Plan A engineering claim: unmodified serving stacks are often *deterministic enough* once you freeze the right metadata — so “looks like noise” is usually non-invariance (reduction trees), not irreducible chaos.

Plan A numerics: close joshc’s “~1% looks like error” channel

Non-invariance (common)	True nondeterminism (rare)
Same math, different FP reduction trees	atomicAdd races (some INT dequant)
(batch / kernel / SKU) → different bits	Warp-scheduler order changes
Deterministic if metadata fixed	Avoid / ban those kernels for audit

Record: SKU · weights · parallel topology · software stack · batch size / step

Then recompute on same SKU, or CPU-emulate tensor-core + SFU rounding (cross-gen).

No need for slow “determinism flags” on the live path — audit via metadata + replay.

Contrast: DiFR = statistical slack; VerInf = don’t require float bit-exact (Û absorbs).

arXiv:2606.00279 — bit-exact inference verification (Cankaya).

- Unmodified vLLM/HF are usually **deterministic** without special determinism flags; “noise” is mostly **non-invariance** (reduction trees), not atomics.
- CPU emulator can match A100/L40/H100 intermediates when SKU + kernel path are modeled.
- **Metadata checklist:** SKU · weights · parallel topology · software stack · per-forward batch size.
- **DiFR:** if you cannot field bit-exact, keep statistical slack — but then budget the covert channel explicitly (joshc’s warning).

> HOW THE PIECES LOCK
>
> Tap → I/O commitments · Bit-exact (+ metadata) → close stego slack · AE recompute (Plan A) evaluates samples. §§6–8 swap in ZK at the evaluate step only.

PLAN B · EXACT-CIRCUIT ZK

6. zkLLM — prove the integer circuit ran

arXiv:2404.16109 · Sun, Li, Zhang · ACM CCS 2024 · code

Who

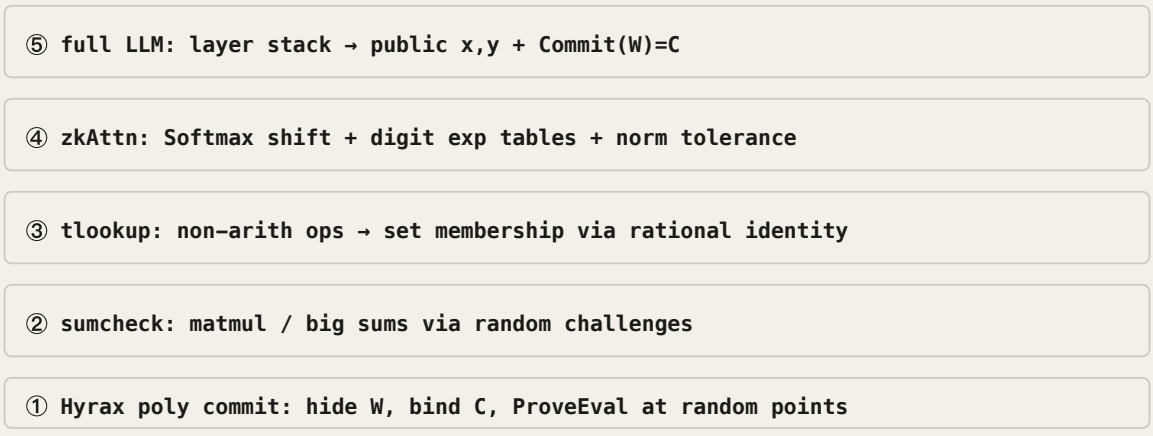
Haochen Sun, Jason Li, Hongyang Zhang — first widely cited exact-circuit ZK system for LLM inference at non-toy scale (~13B).

Maturity

Published system + limited scale. Spot-check engine, not "every API request." Integer/quantized circuit ≠ arbitrary float serving bit-exact.

Statement: there exist weights W matching public commitment C such that the public architecture, run as an *integer circuit* on public x, yields public y — without revealing W or activations.

Lineage: One Plan B evaluate backend under Cankaya's challenge loop. Precursor to FlashProve (§7, same exact-ish receipt idea at larger scale) and contrast to VerInf (§8, unexplained-info score). Complements Plan A when the AE cannot see weights.



zkLLM composition: verifier never re-runs the model — only checks random algebraic relations against commitments.

Numbers (paper anchors)

ITEM	ROUGH PUBLIC FIGURE
Scale	~13B (OPT / LLaMA-2 class)
Prove time	~1–15 min (CUDA, setting-dependent)
Proof size	< ~200 KB
Verify	~1–3 s
Commit scheme	Hyrax (no trusted setup)

Guts (enough to not handwave)

Recipe B in §0c (§§15–16): Hyrax PCS on elliptic-curve discrete log, not Merkle+RS. Names below are zkLLM-specific gadgets on that floor.

- **Hyrax:** see §0c ·16 — binds W into C; ProveEval at random points.
- **Sumcheck:** big matmuls / sums → interactive algebraic checks (see §0c sumcheck).
- **tlookup:** non-arithmetic ops → prove secret tensor entries lie in a public table (see §0c lookups).
- **zkAttn:** Softmax shift shrinks exp domain; don't prove exact log-sum-exp — only a **tolerance band** on row sums (fine for execution receipts; dangerous if bolted onto a manipulable unexplained-info score).

What it does *not* prove

- Bit-exact match to H100/B200 float serving (that's Plan A / bit-exact paper).
- Anything about training — only a forward pass receipt.
- Frontier 100B+ scale in the published system (see FlashProve / VerInf for larger demo with a different statement).

> CHEAT INTUITION
>
> Swap whitelist model → fails vs C. Abuse Softmax tolerance → can still Accept under loose bands. Float nondeterminism is mostly irrelevant because you never claimed float equality.

PLAN B · HASH-BASED PI AT 400B

7. FlashProve — commit-and-prove inference (Anthropic, 2026)

anthropic.com/research/provable-inference · github.com/anthropics/FlashProve · tech report + Lean + Apache-2.0 code · Nov 2026

Who

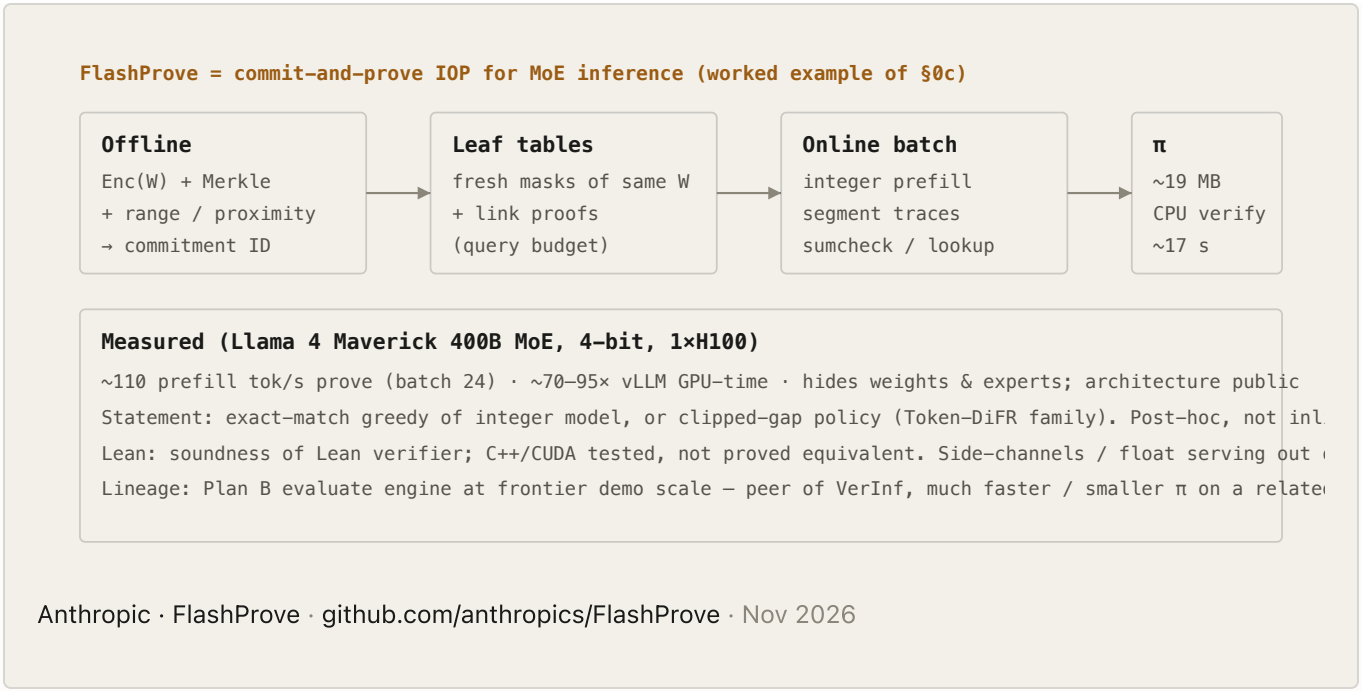
Roy Rinberg, Alex Allain (Anthropic) with Chiesa / Christ / Dalal / Fenzi / Mayer / Weissenberg (EPFL, MIT, StarkWare, Succinct…). Frontier Red Team artifact toward "inference verification."

Maturity

Working open-source prototype at Llama-4-Maverick 400B MoE on 1×H100. Lean proves protocol properties; C++/CUDA are AI-derived from Lean and *tested*, not proved equivalent. Research release, not a serving product.

Statement (exact match): committed integer weights, on a public token window, produce the claimed greedy tokens on a checked range — without revealing weights, activations, or which MoE experts ran. Architecture is public. Serving float / sampled decoding can diverge; production-shaped use is post-hoc prove under a *clipped-gap* policy (Token-DiFR family).

Lineage: Plan B evaluate engine under Cankaya's challenge loop — after zkLLM (§6), before VerInf (§8). Concrete application of §0c recipe A (hash/IOP). Separately, Anthropic's RSP "inference verification" goal was also met by a non-ZK path (sign traffic + recompute on a trusted box) — FlashProve is the cryptographic sibling.



Anthropic · FlashProve · github.com/anthropics/FlashProve · Nov 2026

How it uses §0c (map the names)

§0C PIECE	FLASHPROVE INSTANCE
RS + random mask	Weight / trace rows encoded over ~64-bit prime field; query budget t
Merkle	Columns are tree leaves; commitment ID hashes root + config + OOD answers
Sumcheck (~2k / req)	Matmuls, folds, lookup reductions → linear claims
Lookup	Nonlinearities on small tables (~2^{16}); range-ish checks
Proximity + OOD	List-decoding regime; pin one nearby codeword
Fiat–Shamir (DAG)	Per-request chains join for one shared weight opening per batch
Leaf / link proofs	Fresh encodings of same W so privacy budget doesn't wear out after one batch

Numbers (public release)

ITEM	ROUGH FIGURE
Model	Llama 4 Maverick 400B MoE (~17B active / token), 4-bit integer
Context proved	≤ 1024 tokens / request
Prove	~110 tok/s on 1×H100 (batch 24); ~70–95× vLLM GPU-time
Proof size / verify	~19 MB · ~17 s on 24-core CPU

ITEM	ROUGH FIGURE
Offline commit	`~39 min once; ~6.5 MB public material; ~16M requests / commitment class`

What it does *not* prove

- Bit-exact float serving (integer greedy / clipped gap only).
- Hidden architecture; that the committed model is “the good one” (trust the commitment ID’s publisher).
- That hardware isn’t also training on the side (Claim A residual — PI ≠ pause).
- Timing side-channels; contexts beyond 1k in the release.

> READ WITH §0C OPEN
>
> If §7 still feels like jargon, go back to §0c and re-read the tool whose *problem statement* you cannot repeat in a full sentence — usually sumcheck, lookup, or proximity / list decoding — then return here.

PLAN B · UNEXPLAINED-INFO ZK

8. VerInf — bound leftover bits, skip float bit-exact

Petrie 2026 draft · github.com/JamesPetrie/VerInf · FLI / FlexHEG author

Who

James Petrie — same hardware-governance thread as FlexHEG. VerInf is the "replace the recompute cluster with a ZK evaluate engine" instance at frontier demo scale.

Maturity

Working draft + large demo, not production audit tooling. Interactive, expensive proofs meant for rare high-stakes spot checks when both parties have serious compute.

Statement: a committed integer model explains a (possibly hidden) token stream well enough that leftover unexplained information is ≤ a public bound Û bits/token — without revealing weights, activations, or plaintext I/O.

Lineage: Other Plan B evaluate backend under Cankaya. Designed precisely because Plan A bit-exact and zkLLM-style exact equality both get brittle or tiny-scale at frontier serving.

VerInf statement (not bit-exact equality)

Committed integer model M

predicts hidden token stream
well enough that leftover
info ≤ public Û bits/token
ZK: no weights / acts / x,y

Demo anchor (Petrie 2026)

Llama-4-Maverick ~400B MoE
1000 tokens · Û ≈ 0.88 b/tok
π ≈ 94 GB · prove ~14h · verify ~18h
spot-check scare, not every request

vs zkLLM

zkLLM: prove exact integer circuit output y. Softmax via tables + tolerance. Scale ~13B, tiny π.
VerInf: prove predictive explanation bound. Deliberately skips float bit-exact. Scale ~400B, huge π.
Both are Cankaya Plan B evaluate engines — after taps/challenge exist. Plan A stays the near-term default.

VerInf — unexplained-info ZK at frontier demo scale.

Why change the statement

- Float nondeterminism + MoE/serving stacks make “y must equal AE recompute” expensive to field at 400B.
- Exact-circuit ZK at 13B already hurts; naively scaling the zkLLM statement is the wrong lever.
- Û is an auditable score: too large ⇒ looks like wrong model / stego / garbage; threshold is a policy knob.

Demo anchors (draft)

ITEM	ROUGH FIGURE
Model	Llama-4-Maverick ~400B MoE (commit all experts/layer)
Sequence	1000 tokens (500+500), fully hidden in the demo
Û	`≈ 0.88 bits/token (~95% of vocab info explained)`
Prove / verify	`~14 h prove · ~94 GB π · ~18 h verify (single high-end box class)`
Witness stream	`~7 TB streamed · ~84 GB peak RAM`

vs other evaluate engines (cheat table)

CHEAT	PLAN A (§5)	ZKLLM (§6)	FLASHPROVE (§7)	VERINF (§8)
Swap whitelist model	recompute mismatch	fails vs C	fails vs commitment	Û blows up
Float nondeterminism	pain; bit-exact / metadata	proves circuit, not float replica	integer / clipped-gap; float can diverge	slack → Û
Softmax / gap slack abuse	if loose match, channel	still Accept possible	clipped-gap can be loose if C/θ large	forbidden design goal; unique shift

> COST INTUITION
>
> Spot checks, not every request. Plan A ≈ recompute GPU-time. zkLLM-13B ~minutes + tiny π. FlashProve-400B ~two-order GPU overhead, ~19 MB π. VerInf-400B = hours + tens of GB — rare audits when secrecy of weights beats AE plaintext.

HARDWARE TRACK · PARALLEL TO §§5–8

9. FlexHEG trilogy — especially Part II

Tech 2506.03409 · Overview 2506.15093 · Intl 2506.15100 · flexheg.com · 2025

Who

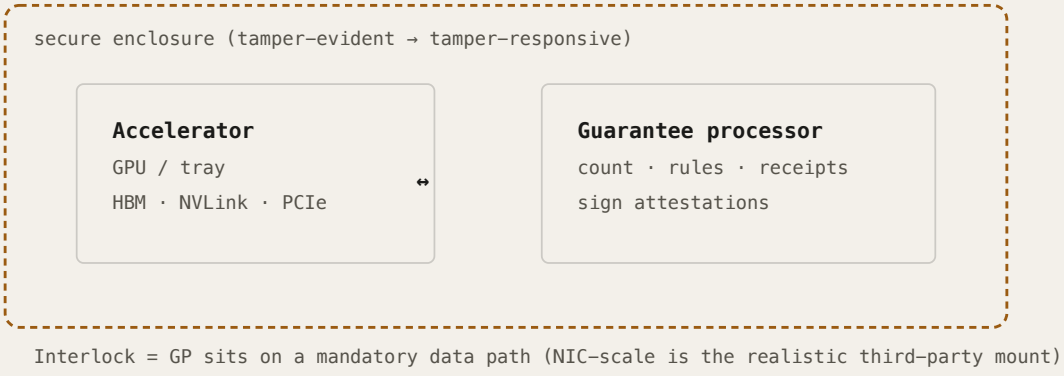
James Petrie, Onni Aarne, and collaborators (FLI / ARIA-linked FlexHEG program). Hardware-enabled governance design for accelerators.

Maturity

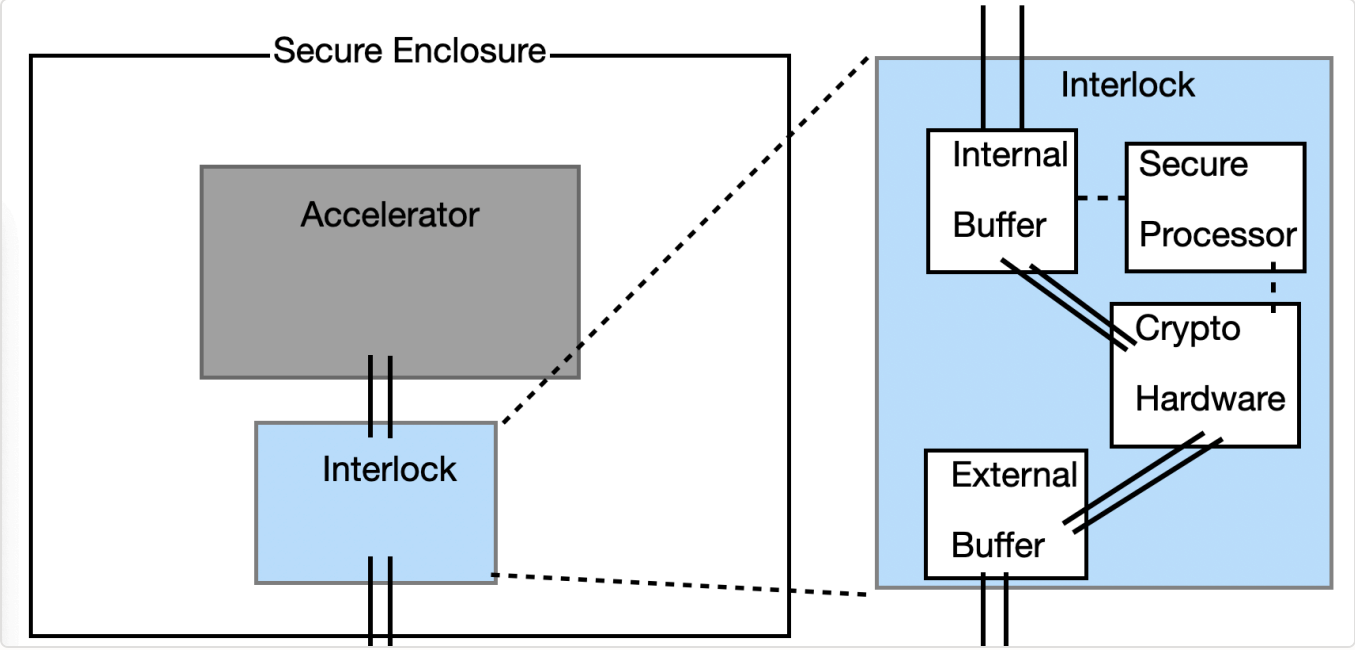
Blueprint + early demos (enclosure / metering ideas). **Not** mass-deployed product silicon. Grant-funded engineering (incl. partners building tap/wipe primitives).

FlexHEG = Flexible Hardware-Enabled Guarantees: a **guarantee processor** that locally enforces / attests rules, plus a **secure enclosure** that makes bypass expensive. Rules should be updatable under multi-party authorization.

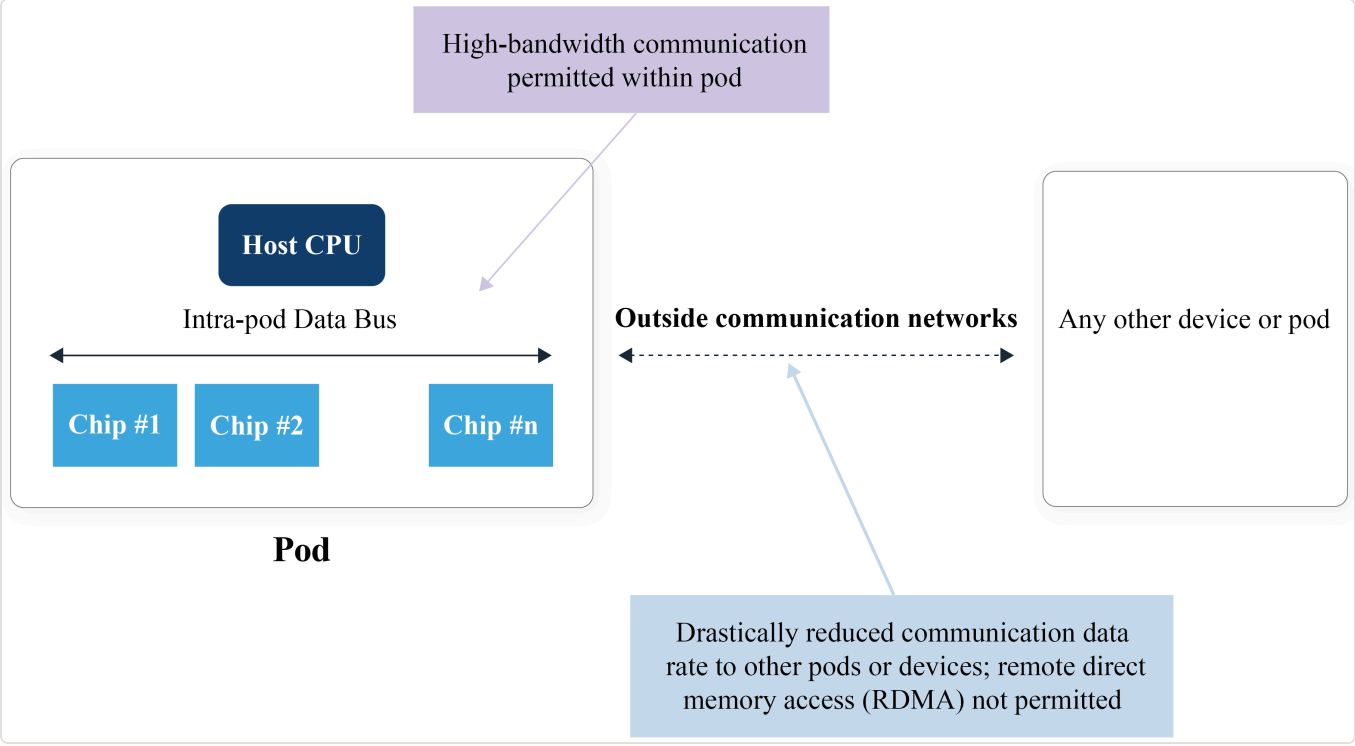
Lineage: After the near-term retrofit path (§§5–8): moves Claim A from Shavit-style after-the-fact snapshots toward *active* guarantees (FLOP caps, licenses, interlocks). Near-term retrofit path often emphasized: NIC / scale-out interlock before in-die changes. Complements Cankaya's facility-level taps (different layer).



FlexHEG dual core: local notarization + physical hardness. Privacy: prove compliance without shipping weights abroad.



Interlock intuition: put the guarantee processor on a path data must cross.



Fixed-set pod idea: fast links inside a declared set; starved RDMA outside — a governance-shaped topology.

Integration ladder (speed vs hardness)

MOUNT	DEPLOY SPEED	SECURITY	WHO CAN SHIP IT
In-die / PCB redesign	slow (multi-year silicon + fleet churn)	strongest	incumbent vendors
Retrofit modules	medium	medium (detach risk)	third parties possible

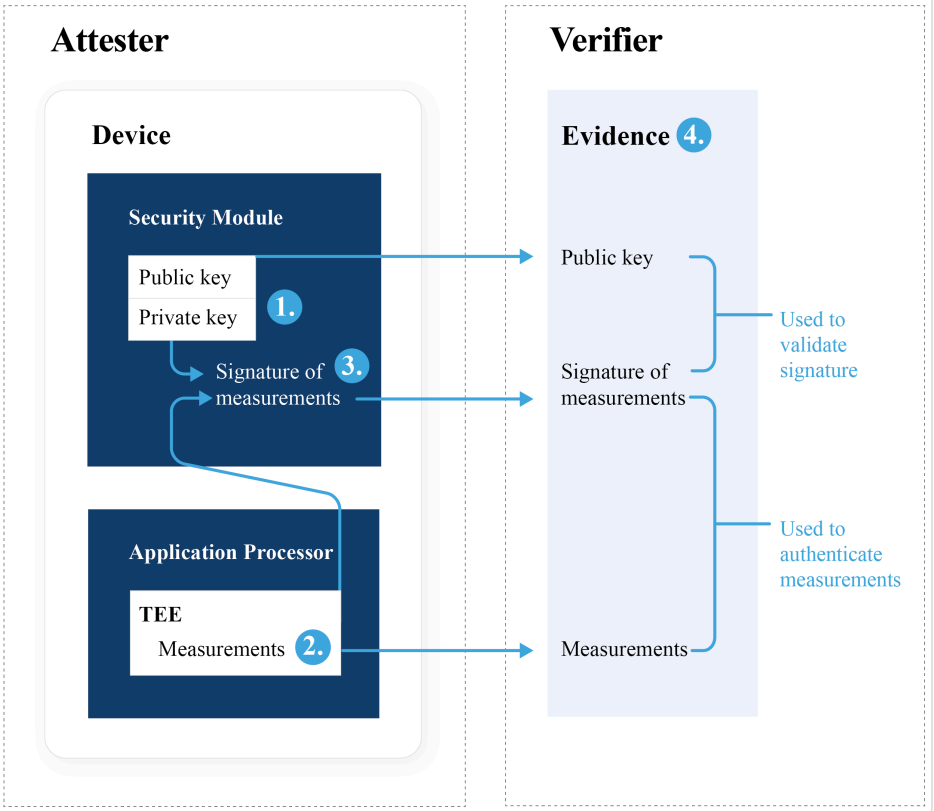
MOUNT	DEPLOY SPEED	SECURITY	WHO CAN SHIP IT
Firmware-only	faster	weaker vs physical	needs signed secure boot
Pure software	fastest	weakest	anyone — not a treaty root

1. A device-specific **Public and Private key pair** are created on the device, ideally using a dedicated **Security Module.** The public key is shared externally.

2. **Measurements** (e.g. the specific firmware loaded onto the device) are collected, ideally using a **TEE**. Using a **TEE** means measurements are only observable and shareable by the **Attester.**

3. The **Security Module** uses the **Private key** to create a **"signature" of measurements** taken from the device. This will be used to prove that the **measurements** collected came from the device.

4. Using the **evidence** provided by the **Attester,** the **Verifier** can now validate that the **measurements** are genuine.



Attestation mental model — local measurement → signed statement → remote verify.

CRYPTO HORIZON · TRAINING

10. Peigné–Lefebvre et al. — ZK verification of frontier *training*

arXiv:2606.05433 · 2026 · GPAI Policy Lab / FAR.AI · Sorbonne/LIP6

Who

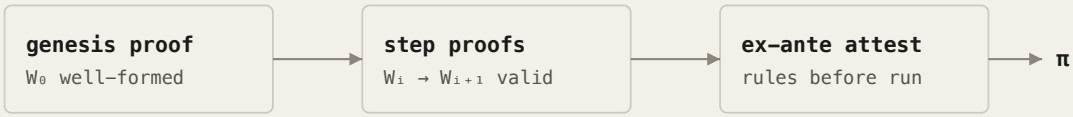
Pierre Peigné-Lefebvre, Ky Nguyen, and coauthors — argument that ZK verification of frontier *training* is possible in principle via a BF16/FP32-aware zkVM.

Maturity

Horizon. Authors are explicit: no working end-to-end prototype; rough ~36 months to a deployable PoC vs 6–10 years for custom verification silicon; long open-problem list.

Idea: make Shavit's transcript story cryptographically binding — genesis / step / optional ex-ante proofs without revealing the proprietary run.

Lineage: Different problem than §§5–8 (those audit *inference* whitelist forward passes). This is the crypto endgame of classical *training* monitoring — §0c recipe C (zkVM / recursion / folding). Does not replace near-term FlexHEG/Cankaya work; clarifies what "strong" would mean if telemetry stays porous.



vs Shavit

Shavit: heuristic / sampling recompute of a claimed transcript

ZK: cryptographic proof over BF16/FP32 training steps in a zkVM — stronger claim, no E2E prototype yet

Authors: ~36 months to deployable PoC; custom verify silicon 6–10 years; 13 open problems listed

Crypto endgame of the Shavit idea. Treat as research horizon, not near-term pause tooling.

SYNTHESIS

11. Putting the stack in one breath

What an outsider should be able to repeat after this deck.

```
Claim A = verify no covert frontier training (≠ TEE model-identity or
confidential-cloud claims B/C). Wasil is the method menu. Shavit is the
classical chip+transcript blueprint. Rahman measures how far NVML train↔infer
detectors survive disguise (closes Shavit's lookalike gap). joshc red-teams cheap
facility controls against covert RL.

Cankaya is the near-term low-trust retrofit: NS-first taps feed commitments;
Plan A = AE precise replay (bit-exact metadata; DiFR if needed). Plan B swaps
the evaluate step for ZK — zkLLM (~13B exact integer circuit), FlashProve
(~400B MoE greedy / clipped-gap, ~110 tok/s, ~19 MB π), or VerInf
(~400B unexplained-info bound Û). §0c is the complete ZK primer for this deck
(shared floor + hash/IOP recipe A + Hyrax/PCS recipe B + statement choice + zkVM rec
each tool tagged with which direction uses it. FlexHEG is the parallel hardware-enfo
track (active caps / interlocks), not a substitute for taps+evaluate. Peigné
sketches ZK *training* via zkVM as a longer crypto horizon. Almost none of this is m
empirics, and prototypes.
```

Open problems / next-step research (what to do after this deck)

These are not "unsolved math curiosities" — they are the load-bearing gaps between today's blueprints and a credible pause. Several track cruxes raised against Cankaya's retrofit (precise replay, sampling economics, perimeter, rare covert adapters, Plan A→B transition, adjudication).

#	PROBLEM	WHY IT BLOCKS	CONCRETE NEXT STEP
1	**Precise replay vs softer accept**	Plan A soundness/completeness hinge on bit-exact (or DiFR slack). Unknown: residual mismatch rate across SKU/driver/batch, and FP cost of chasing zero mismatch vs budgeting a channel.	Medium-scale empirics (e.g. ~70B) across heterogeneous GPUs/drivers: exact-match rate vs DiFR-style accept; metadata necessity set; cost/latency of each policy.
2	**Sampling economics**	Assurance = soundness × sample rate. Appendix-style Fermi needs a real BoM: can Plan A / zkLLM / VerInf hit ~1% overhead at a stated catch probability?	Publish cost curves: challenge rate → catch prob → $/request for AE recompute vs zkLLM-13B vs VerInf-400B-class proofs.

#	PROBLEM	WHY IT BLOCKS	CONCRETE NEXT STEP
3	**Perimeter / NS-only residual**	NS-first taps leave EW copper and off-perimeter compute undertapped. Bar: residual outside the fence can't do consequential covert work at high confidence.	NS-only red-team budget (joshc-style local RL / DiLoCo): how many covert FLOPs survive; optional interconnect detectors (Fowler-style) as complement.
4	**Rare-trigger / thin adapters**	Forward-pass recompute can match a whitelist while a rarely activated LoRA / tool path does harm — identity ≠ all governance goals.	Red-team: memory probes, NVML anomaly, wipe/accounting, inspector agents under disclosure bounds; measure miss rate for sparse triggers.
5	**Train↔infer under disguise**	Rahman: hard disguises degrade NVML classifiers. joshc: RL rollouts look like serving. Soft labels alone won't carry Claim A.	Combined channel study: NVML + interconnect bytes + tap commitments under adaptive evade; report ROC under the hardest loop, not only benign accuracy.
6	**Plan A → Plan B transition**	Plaintext AE burns confidentiality; ZK restores it but is immature/expensive. Stakeholders need a migration path, not a cliff.	Timeline + dual-run design: which challenge classes stay on AE, which move to zkLLM / VerInf / FlashProve as prove-time falls; joint ops with multiple evaluate engines.
7	**Physical security of AE / taps**	Low-trust Plan A assumes rival TCBs + continuous physical monitoring. If forgeable silently, assurance collapses.	Red-team / OSINT: diodes, FPGA wardens, SCIFs at scale, hosting foreign TCBs on sensitive weights — map failure modes to FlexHEG hardware track.
8	**Adjudication & consequences**	Technical Accept/Reject ≠ treaty breach. Without an escalation ladder + false-positive budget, either everything is excused or everything detonates.	Design CBRN-like ladder: anomaly → joint review → deeper disclosure → supervised inspection → predetermined response; name who adjudicates.
9	**Hardware enforcement demos**	FlexHEG remains the long enforcement story; near-term needs NIC/enclosure interlocks with honest overheads and multi-party rule updates.	Ship a NIC-level license/FLOP-cap prototype; measure overhead; stress single-vendor root-of-trust objections.

> HOW TO READ THIS LIST
>
> **1–5** are mostly empirical / systems (can move in months). **6–7** mix engineering with deployment politics. **8** is institutional. **9** / §9 FlexHEG is the parallel hardware track. A credible pause needs progress on several in parallel — closing only crypto or only taps is not enough.

> PRIMARY SOURCES (PUBLIC)

Wasil 2408.16074 · Shavit 2303.11341 · joshc LW · FlexHEG flexheg.com · Rahman 2606.19262 · Cankaya overview LW · taps Substack · bit-exact 2606.00279 · zkLLM 2404.16109 · VerInf GitHub · FlashProve Anthropic / code · DiFR 2511.20621 · Peigné 2606.05433